

第一章

1 从数据结构的角度看，以下哪一项不属于其核心组成部分（C）

- A. 数据的逻辑结构
- B. 数据的存储结构
- C. 数据元素的内容
- D. 在数据上定义的运算集合

2 数据的逻辑结构分类中，不包含以下哪类？（B）

- A. 集合结构
- B. 链式结构
- C. 树形结构
- D. 图状结构

3 顺序存储结构的核心特点不包括（D）

- A. 存储密度大，空间利用率高
- B. 插入、删除操作需移动大量元素
- C. 逻辑上相邻的元素物理位置必相邻
- D. 通过指针字段表示元素间的逻辑关系

4 从存储结构角度看，链式结点必须包含的部分是（B）

- A. 关键字
- B. 数据域和指针域
- C. 索引项
- D. 散列地址

5 索引存储结构中，索引项的标准组成形式是（A）

- A. 关键字 + 地址
- B. 数据项 + 指针
- C. 关键字 + 数据元素
- D. 物理地址 + 逻辑地址

6 散列存储（Hash 表）中，确定结点存储地址的核心依据是（B）

- A. 结点的逻辑位置
- B. 结点的关键字
- C. 索引表的索引项
- D. 结点的物理偏移量

7 抽象数据类型（ADT）的核心特性不包含（C）

- A. 抽象性
- B. 封装性
- C. 继承性
- D. 数学模型化

8 算法“正确性”的四个层次中，最高级别的是 (D)

- A. 程序无语法错误
- B. 几组常规输入数据输出正确
- C. 苛刻、刁难性输入数据输出正确
- D. 所有合法输入数据输出正确

9 算法与程序的区别描述错误的是 (C)

- A. 算法必须满足有穷性，程序可能存在死循环
- B. 算法的指令可非机器执行，程序指令必须机器可执行
- C. 算法着重描述实现细节，程序着重描述解决思路
- D. 算法是有限操作序列，程序是算法的计算机语言实现

10 若算法的时间耗费函数为 $T(n) = 5n^3 + 3n^2 + 7n + 2$ ，则其渐近时间复杂度为 (A)

- A. $O(n^3)$
- B. $O(5n^3)$
- C. $O(n^2)$
- D. $O(n)$

11 下列时间复杂度中，增长率最低的是 (A)

- A. $O(\log_2 n)$
- B. $O(n)$
- C. $O(n \log_2 n)$
- D. $O(n^2)$

12 分析程序段的时间复杂度：

```
for(int i=1; i<=n; i*=2)
    for(int j=1; j<=i; j++)
        sum++;
```

其时间复杂度为 (B)

- A. $O(\log_2 n)$
- B. $O(n)$
- C. $O(n \log_2 n)$
- D. $O(n^2)$

13 递归函数 $\text{func}(n)$ 的时间复杂度为 (C)

```
int func(int n) {
    if(n <= 1) return 1;
    int sum = 0;
    for(int i=0; i<n; i++) sum++;
    return func(n/2) + func(n/2) + sum;
}
```

- A. $O(n)$
- B. $O(\log_2 n)$
- C. $O(n \log_2 n)$

D. $O(n^2)$

14 数据的物理结构（存储结构）指的是（B）

- A. 数据元素之间的逻辑关系
- B. 数据元素及其关系在计算机中的存储表示
- C. 定义在数据上的运算集合
- D. 数据对象的有限集合

15 下列属于非线性逻辑结构的是（C）

- A. 栈
- B. 队列
- C. 树
- D. 顺序表

16 顺序存储结构中，若基地址为 base，结点大小为 size，则第 i 个结点（从 1 开始计数）的存储地址为（B）

- A. $\text{base} + i \times \text{size}$
- B. $\text{base} + (i-1) \times \text{size}$
- C. $\text{base} + \text{size} / i$
- D. 由指针域计算得出

17 链式存储结构的存储密度低于顺序存储结构的根本原因是（B）

- A. 链式存储元素物理位置不连续
- B. 链式存储需额外存储指针域（链接信息）
- C. 链式存储插入删除更灵活
- D. 链式存储逻辑结构与物理结构无关

18 抽象数据类型（ADT）的具体实现主要取决于（C）

- A. 用户对逻辑结构的理解
- B. 算法的时间和空间效率要求
- C. 所采用的存储结构
- D. 编程语言是否支持面向对象特性

19 算法的“健壮性”具体指（C）

- A. 算法执行时间有限（有穷性）
- B. 算法逻辑清晰、易读易调试（可读性）
- C. 输入非法数据时能适当处理，不死机或输出异常结果
- D. 能完成期望的功能（正确性）

20 分析三重循环的时间复杂度：

```
for(int i=0; i<n; i++)  
    for(int j=i; j<n; j++)  
        for(int k=0; k<j-i+1; k++)  
            count++;
```

其时间复杂度为 (C)

- A. $O(n)$
- B. $O(n^2)$
- C. $O(n^3)$
- D. $O(n\log_2 n)$

21 数据结构的形式定义 $DS = (D, S)$ 中, S 表示 (B)

- A. 数据元素的有限集合
- B. 数据元素之间关系的有限集合
- C. 数据的存储结构
- D. 数据的运算集合

22 下列操作中, 不属于数据基本运算的是 (C)

- A. 查找
- B. 排序
- C. 封装
- D. 插入

23 索引存储结构的核心优势是 (C)

- A. 存储密度高
- B. 插入删除操作无需移动元素
- C. 查找速度快
- D. 空间利用率高

24 下列逻辑结构中, 最不适合采用顺序存储结构实现的是 (C)

- A. 线性表
- B. 完全二叉树
- C. 稀疏图
- D. 队列

25 本课程中, 算法的主要描述方式是 (D)

- A. 自然语言
- B. 流程图
- C. 数学符号语言
- D. C++ 函数模板 / 函数

26 空间复杂度 $S(n) = O(g(n))$ 的准确含义是 (B)

- A. 算法所需存储空间等于 $g(n)$
- B. 算法所需存储空间的增长率与 $g(n)$ 的增长率相同
- C. 算法所需辅助空间为 $g(n)$
- D. 算法为原地工作 (辅助空间为常量)

27 递归函数 $func(n)$ 的时间复杂度为 (A)

```
int func(int n) {
```

```

    if(n <= 3) return 1;
    return func(n/3) + func(n/3) + func(n/3);
}

```

- A. $O(n)$
- B. $O(\log_3 n)$
- C. $O(n \log_3 n)$
- D. $O(3^n)$

28 逻辑结构与存储结构的关系描述正确的是 (D)

- A. 逻辑结构决定存储结构
- B. 存储结构决定逻辑结构
- C. 逻辑结构与存储结构相互独立，无关联
- D. 逻辑结构通过存储结构实现，存储结构依赖逻辑结构

29 顺序存储结构最适用于哪种逻辑结构？ (A)

- A. 线性结构
- B. 树形结构
- C. 图状结构
- D. 集合结构

30 数据项的定义是 (B)

- A. 数据的基本单位（程序中作为整体处理）
- B. 具有独立意义的不可分割的最小单位
- C. 具有相同性质的数据元素的集合
- D. 计算机中可操作的所有符号的总称

31 算法的“有限性”指的是 (B)

- A. 算法的输入输出数量有限
- B. 算法的执行步骤有限且每个步骤可在有限时间内完成
- C. 算法所需存储空间有限
- D. 算法的时间复杂度为有限值

32 分析程序段的空间复杂度：

```

void func(int n) {
    int** mat = (int**)malloc(n * sizeof(int*));
    for(int i=0; i<n; i++)
        mat[i] = (int*)malloc(n * sizeof(int));
    // 释放内存（省略）
}

```

其空间复杂度为 (C)

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(\log_2 n)$

- 33 抽象数据类型 (ADT) 的 “封装性” 具体指 (B)
- A. 强调实体的本质特征和外部接口
 - B. 将实体的外部特性与内部实现细节分离, 隐藏内部细节
 - C. 由基本数据类型或其他构造类型组成
 - D. 支持多态操作 (同一接口多种实现)
- 34 下列属于数据存储结构的是 (B)
- A. 线性结构
 - B. 链式结构
 - C. 树形结构
 - D. 图状结构
- 35 散列存储结构最可能面临的问题是 (C)
- A. 插入删除操作复杂
 - B. 存储密度过低
 - C. 哈希冲突 (不同关键字计算出相同地址)
 - D. 查找依赖索引表, 效率低下
- 36 算法分析中, 时间复杂度与空间复杂度的关系是 (B)
- A. 相互独立, 无关联
 - B. 对立统一 (往往优化一方会牺牲另一方)
 - C. 时间复杂度优先于空间复杂度
 - D. 空间复杂度优先于时间复杂度
- 37 递归函数 `func(n)` 的时间复杂度为 (B)
- ```
void func(int n) {
 if(n == 0) return;
 for(int i=0; i<n; i++)
 for(int j=0; j<n; j++)
 printf("*");
 func(n-1);
}
```
- A.  $O(n^2)$
  - B.  $O(n^3)$
  - C.  $O(n^4)$
  - D.  $O(2^n)$
- 38 数据对象的准确定义是 (C)
- A. 计算机中可操作的所有符号的总称
  - B. 数据的基本单位 (结点或记录)
  - C. 具有相同性质的数据元素的集合 (数据的子集)
  - D. 数据项的有限集合

39 顺序存储结构中，存储单元的地址特点是 (A)

- A. 一定连续
- B. 一定不连续
- C. 部分连续，部分不连续
- D. 由元素逻辑关系决定是否连续

40 下列关于时间复杂度计算的说法，错误的是 (D)

- A. 忽略低阶项 (如  $n^2+5n$  中的  $5n$ )
- B. 保留最高阶项 (如  $3n^3+2n^2$  中的  $3n^3$ )
- C. 去除最高阶项的系数 (如  $3n^3$  变为  $n^3$ )
- D. 保留所有加法常数 (如  $n+3$  中的  $3$ )

## 第 2 章

1 已知线性表的顺序存储结构中，每个元素占  $k$  个单元，逻辑上的第 1 个元素为  $a_1$ ，其地址为  $\text{loc}(a_1)$ ，则第  $i$  个元素的地址  $\text{loc}(a_i)$  计算正确的是 (C)

- A.  $\text{loc}(a_1) + i \times k$
- B.  $\text{loc}(a_1) + (i+1) \times k$
- C.  $\text{loc}(a_1) + (i-1) \times k$
- D.  $\text{loc}(a_1) + (n-i) \times k$

2 对于带头结点的单链表，若要在第  $i$  个元素 ( $1 \leq i \leq n$ ) 之前插入新结点，则新结点的前驱节点是 (B)

- A.  $i$
- B.  $i-1$
- C.  $i+1$
- D.  $n$

3 顺序表中，在长度为  $n$  的线性表第  $i$  个位置 ( $1 \leq i \leq n+1$ ) 插入元素时，需向后移动的元素个数为 (B)

- A.  $n-i$
- B.  $n-i+1$
- C.  $i$
- D.  $i-1$

4 单链表的存储密度小于 1 的根本原因是 (B)

- A. 数据元素不连续存储
- B. 每个结点除数据域外还需额外存储指针域
- C. 插入删除操作无需移动元素
- D. 无法实现随机存取

5 双向链表中，若要删除结点  $p$  (非头结点、非尾结点)，正确的操作顺序是 (B)

- A.  $p \rightarrow \text{next} = p \rightarrow \text{prior}; p \rightarrow \text{prior} = p \rightarrow \text{next}; \text{free}(p)$
- B.  $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}; \text{free}(p)$
- C.  $\text{free}(p); p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}$
- D.  $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}; \text{free}(p); p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}$

6 循环链表与普通单链表的核心区别是 (B)

- A. 循环链表必须带头结点
- B. 终端结点的指针域指向表头而非 NULL
- C. 循环链表支持随机存取
- D. 循环链表的长度不可变

7 假设线性表的位置编号从 1 开始，其抽象数据类型 (ADT) 中，Insert 操作的 position 合法取值范围是 (C)



- A.  $1 \leq \text{position} \leq \text{Length}()$
- B.  $0 \leq \text{position} \leq \text{Length}()$
- C.  $1 \leq \text{position} \leq \text{Length}() + 1$
- D.  $0 \leq \text{position} \leq \text{Length}() + 1$

8 顺序表的 Delete 操作平均时间复杂度为 (B)

- A.  $O(1)$
- B.  $O(n)$
- C.  $O(\log n)$
- D.  $O(n^2)$

9 单链表中按序号查找第  $i$  个结点 ( $1 \leq i \leq n$ ), 最坏情况下的时间复杂度为 (B)

- A.  $O(1)$
- B.  $O(n)$
- C.  $O(\log n)$
- D.  $O(n^2)$

10 对于带头结点的空循环链表, 以下描述正确的是 (C)

- A. 头指针为 NULL
- B. 头结点的 next 指针指向 NULL
- C. 头结点的 next 指针指向自身
- D. 不存在头结点

11 已知顺序表的  $\text{maxSize}=10$ , 当前  $\text{count}=8$  (元素个数为 8), 执行 Insert (9, e) 操作 (在第 9 个位置插入 e), 需移动的元素个数为 (A)

- A. 0
- B. 1
- C. 2
- D. 8

12 双向链表的结点结构中, 包含的指针域个数为 (B)

- A. 1
- B. 2
- C. 3
- D. 4

13 线性表的逻辑结构特点不包括以下哪一项 (D)

- A. 存在唯一的 “第一个” 数据元素
- B. 存在唯一的 “最后一个” 数据元素
- C. 除首尾元素外, 每个元素有且仅有一个前驱和一个后继
- D. 数据元素的物理位置必须相邻

14 采用单链表头插法 (每次插入到头结点之后) 建表, 输入数据序列为 1、2、3、4, 则链表中元素的逻辑顺序为 (B)

- A.  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
- B.  $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$
- C.  $2 \rightarrow 1 \rightarrow 4 \rightarrow 3$
- D.  $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$

15 顺序表的优点不包括以下哪一项 (D)

- A. 存储结构简单
- B. 存储密度高 (存储密度 = 1)
- C. 支持随机存取操作
- D. 插入、删除操作效率高

16 循环链表中, 若用尾指针 rear 表示链表, 访问表头结点 (头结点) 的操作是 (B)

- A. Rear
- B. rear->next
- C. rear->next->next
- D. head

17 在顺序表中实现删除操作 Delete(i, &e) 时, 以下说法正确的是 (C)

- A. 删除位置 i 的合法范围是  $1 \leq i \leq \text{Length}() + 1$
- B. 删除操作的时间复杂度为  $O(1)$
- C. 需将第 i+1 至第 Length() 个元素依次向前移动一个位置
- D. 若表为空, 则仍可执行删除操作并返回成功

18 单链表中, 若 p 指向某结点, 要删除 p 的直接后继结点, 正确的操作是 (B)

- A. p->next = p->next->next; free (p->next)
- B. Node\* q = p->next; p->next = q->next; free(q)
- C. free(p->next); p->next = NULL
- D. p->next = NULL; free(p->next)

19 存储密度的定义是 (A)

- A. 结点数据本身所占存储量 / 结点结构所占存储总量
- B. 结点结构所占存储总量 / 结点数据本身所占存储量
- C. 数据元素个数 / 分配的存储空间总量
- D. 分配的存储空间总量 / 数据元素个数

20 顺序表中, 删除第 i 个元素 ( $1 \leq i \leq n$ ) 时, 需向前移动的元素个数为 (A)

- A. n-i
- B. n-i+1
- C. i
- D. i-1

21 双向循环链表中, 头结点的 prior 指针指向 (C)

- A. 自身

- B. 首元结点 (第一个数据结点)
- C. 尾结点
- D. NULL

22 对于带头结点的单链表, 判断链表为空表的条件是 (B)

- A. `head == NULL`
- B. `head->next == NULL`
- C. `head->next == head`
- D. `head != NULL`

23 顺序表采用一维数组存储, 数组下标从 0 开始, 线性表中第  $i$  个元素 (逻辑位序,  $1 \leq i \leq n$ ) 对应的数组下标为 (B)

- A.  $i$
- B.  $i-1$
- C.  $i+1$
- D.  $n-i$

24 为避免每次插入都遍历链表, 单链表尾插法建表的核心是需要额外设置 (B)

- A. 头指针
- B. 尾指针
- C. 当前指针
- D. 前驱指针

25 顺序表的 Clear 操作功能是 (B)

- A. 释放顺序表的存储空间
- B. 清空线性表, 将元素个数 `count` 置为 0
- C. 缩小顺序表的最大容量 `maxSize`
- D. 初始化顺序表

26 循环链表的遍历操作中, 终止条件是 (B)

- A. 遍历指针为 NULL
- B. 遍历指针等于头指针
- C. 遍历指针的 `next` 为 NULL
- D. 遍历指针的 `next` 等于尾指针

27 双向链表中, 将结点  $s$  插入到结点  $p$  之后, 正确的操作顺序是 (B)

- A. `s->next = p->next; p->next->prior = s; s->prior = p; p->next = s`
- B. `s->prior = p; s->next = p->next; p->next->prior = s; p->next = s`
- C. `p->next = s; s->prior = p; s->next = p->next; p->next->prior = s`
- D. `p->next->prior = s; s->next = p->next; s->prior = p; p->next = s`

28 以下关于线性表的叙述中, 正确的是 (C)

- A. 线性表的顺序存储结构支持随机访问, 其插入和删除的时间复杂度  $O(1)$
- B. 线性表的链式存储结构中, 各结点的存储空间必须连续

- C. 线性表是一种逻辑结构，其元素之间存在一对一的线性关系
- D. 顺序表能表示空表，链表无法表示空表

29 顺序表的  $\text{maxSize}=10$ ，当前  $\text{count}=10$ （已满），执行  $\text{Insert}(5, e)$  操作，返回的状态是（C）

- A. SUCCESS
- B. ERROR
- C. OVER\_FLOW
- D. OK

30 单链表中按值查找（查找数据域等于  $\text{key}$  的结点），若查找成功，返回的结果是（C）

- A. 结点的数据值  $\text{key}$
- B. 结点的逻辑序号
- C. 首次找到的结点的存储地址（指针）
- D. 所有匹配结点的存储地址

31 循环链表中，若尾指针为  $\text{rear}$ ，将新结点  $s$  插入到表头（头结点之后）的操作是（B）

- A.  $s \rightarrow \text{next} = \text{rear}; \text{rear} \rightarrow \text{next} = s; \text{rear} = s$
- B.  $s \rightarrow \text{next} = \text{rear} \rightarrow \text{next}; \text{rear} \rightarrow \text{next} = s$
- C.  $s \rightarrow \text{next} = \text{rear} \rightarrow \text{next}; \text{rear} \rightarrow \text{next} = s; \text{rear} = s$
- D.  $s \rightarrow \text{next} = \text{rear}; \text{rear} = s$

32 线性表的逻辑结构本质是（B）

- A. 集合结构
- B. 线性结构（一对一关系）
- C. 树形结构
- D. 图状结构

33 顺序表的  $\text{GetElem}$  操作（获取指定位置元素）的时间复杂度为（A）

- A.  $O(1)$
- B.  $O(n)$
- C.  $O(\log n)$
- D.  $O(n^2)$

34 单链表中，若要在结点  $p$ （非头结点）之前插入新结点  $s$ ，正确的做法是（B）

- A.  $s \rightarrow \text{next} = p; p \rightarrow \text{prior} = s$
- B. 先遍历链表找到  $p$  的前驱结点  $q$ ，再执行  $q \rightarrow \text{next} = s; s \rightarrow \text{next} = p$
- C.  $s \rightarrow \text{next} = p; p \rightarrow \text{next} = s$
- D.  $q = p \rightarrow \text{next}; p \rightarrow \text{next} = s; s \rightarrow \text{next} = q$

35 双向链表相比单链表的核心优点是（B）

- A. 存储密度更高
  - B. 可快速访问前驱结点和后继结点（时间复杂度  $O(1)$ ）
  - C. 插入删除操作无需修改指针
  - D. 结构更简单
- 36 线性表 ADT 的 Empty 操作，其功能是 (B)
- A. 返回线性表的长度
  - B. 线性表为空返回 true，否则返回 false
  - C. 返回线性表的首元素
  - D. 返回线性表的尾元素
- 37 顺序表中，当插入位置  $\text{position} = \text{Length}() + 1$  时，新元素插入到 (C)
- A. 表头（第一个位置）
  - B. 表中任意位置
  - C. 表尾（最后一个元素之后）
  - D. 第  $\text{Length}()$  个位置之前
- 38 单链表中设置头结点的主要目的是 (B)
- A. 存储额外的链表信息
  - B. 使表头插入、表尾插入等操作算法统一（无需特殊处理表头）
  - C. 提高数据存取效率
  - D. 标识链表的名称
- 39 循环链表中，若用头指针 head 表示，查找尾结点的操作是 (B)
- A.  $p = \text{head}; \text{while} (p \rightarrow \text{next} \neq \text{NULL}) p = p \rightarrow \text{next};$
  - B.  $p = \text{head}; \text{while} (p \rightarrow \text{next} \neq \text{head}) p = p \rightarrow \text{next};$
  - C.  $p = \text{head} \rightarrow \text{next}; \text{while} (p \rightarrow \text{next} \neq \text{head}) p = p \rightarrow \text{next};$
  - D.  $p = \text{head}; \text{while} (p \neq \text{head}) p = p \rightarrow \text{next};$
- 40 线性表的“同一性”特点是指 (C)
- A. 数据元素的物理存储位置相同
  - B. 数据元素的逻辑顺序与物理顺序一致
  - C. 线性表由同类数据元素组成，每个  $a_i$  属于同一数据对象
  - D. 相邻数据元素之间存在序偶关系  $\langle a_i, a_{i+1} \rangle$
- 41 顺序表的 Delete 操作中，“用 e 返回被删除元素的值”的执行顺序是 (B)
- A. 先移动元素，再获取元素值，最后 count 减 1
  - B. 先获取元素值，再移动元素，最后 count 减 1
  - C. 先 count 减 1，再获取元素值，最后移动元素
  - D. 先 count 减 1，再移动元素，最后获取元素值
- 42 单链表中删除尾结点（长度为 n），最坏情况下的时间复杂度为 (B)
- A.  $O(1)$
  - B.  $O(n)$ （需遍历到倒数第二个结点）

- C.  $O(\log n)$
- D.  $O(n^2)$

43 双向循环链表为空表的条件是 (C)

- A. `head == NULL`
- B. `head->next == NULL`
- C. `head->next == head && head->prior == head`
- D. `head->prior == NULL`

44 线性表的“有穷性”特点是指 (B)

- A. 数据元素的类型有限
- B. 线性表由有限个数据元素组成
- C. 数据元素的存储容量有限
- D. 数据元素的逻辑关系有限

45 要求将顺序表 (元素为 `int` 型) 调整为“左奇右偶”，下列算法中时间复杂度为  $O(n)$ ，空间复杂度为  $O(1)$  的是 (B)

- A. 遍历表，将偶数元素逐一移到表尾
- B. 设置左右指针 (`left=0`, `right=n-1`)，`left` 找偶数、`right` 找奇数，交换两者
- C. 采用排序算法按奇偶性排序
- D. 新建两个顺序表，分别存储奇数和偶数，再合并

46 非循环单链表中，若 `p` 指向某结点，且 `p->next == NULL`，则 `p` 是 (C)

- A. 头结点
- B. 首元结点
- C. 尾结点
- D. 空结点

47 循环链表的约瑟夫问题 ( $n$  个人围成一圈，报数  $m$  出列)，其算法的时间复杂度为 (C)

- A.  $O(n)$
- B.  $O(m)$
- C.  $O(nm)$  (需循环  $n-1$  次，每次遍历  $m$  个结点)
- D.  $O(n^2)$

48 线性表的链式存储结构中，结点内部的存储空间 (A)

- A. 必须连续 (结点是一个结构体，成员地址连续)
- B. 必须不连续
- C. 可连续可不连续
- D. 与其他结点的存储空间必须连续

49 顺序表的存储密度为 (B)

- A. 小于 1

- B. 等于 1（无额外指针开销）
- C. 大于 1
- D. 不确定（与元素类型有关）

50 双向链表中，删除结点 p 的前驱结点 q（q 非头结点），正确的操作是（B）

- A. `q->prior = p; p->prior = q->prior; free (q)`
- B. `p->prior = q->prior; q->prior->next = p; free(q)`
- C. `free(q); p->prior = q->prior; q->prior->next = p`
- D. `p->prior = q->prior; free(q); q->prior->next = p`

### 第 3 章

- 1 栈是操作受限的线性表，其受限的操作位置是 (C)
  - A. 仅栈底插入、仅栈顶删除
  - B. 仅栈顶插入、仅栈底删除
  - C. 仅栈顶插入和删除
  - D. 仅栈底插入和删除
  
- 2 设顺序栈的存储空间为  $\text{ElemType} [\text{MaxSize}]$ ，count 为元素个数，栈满的条件是 (B)
  - A.  $\text{count} == 0$
  - B.  $\text{count} == \text{MaxSize}$
  - C.  $\text{count} == \text{MaxSize} - 1$
  - D.  $\text{top} == \text{MaxSize}$
  
- 3 两个栈共享一维数组  $S [M]$ ，栈底分别在 0 和  $M-1$ ，栈满的标志是 (A)
  - A.  $\text{Top1} == \text{Top2} - 1$
  - B.  $\text{Top1} + \text{Top2} == M$
  - C.  $\text{Top1} == M/2 \ \&\& \ \text{Top2} == M/2$
  - D.  $\text{Top1} == \text{Top2}$
  
- 4 链式栈的栈顶指针指向链表的 ( )，且链式栈 ( ) 栈满问题 (B)
  - A. 尾节点；存在
  - B. 头节点；不存在
  - C. 头节点；存在
  - D. 尾节点；不存在
  
- 5 设有一个栈，元素按给定顺序依次入栈，在入栈过程中可随时执行出栈操作，其入栈序列为 1, 2, 3, 4, 5，下列出栈序列中不可能出现的是 (D)
  - A. 3, 2, 5, 4, 1
  - B. 2, 3, 5, 4, 1
  - C. 2, 1, 3, 4, 5
  - D. 1, 5, 2, 3, 4
  
- 6 设有一个栈，元素按给定顺序依次入栈，在入栈过程中可随时执行出栈操作，其入栈序列为 A, B, C, D, E，以 C 为第一个出栈元素、E 为第三个出栈元素的合法序列有 (B)
  - A. 2 个
  - B. 3 个
  - C. 4 个
  - D. 5 个



7 链式栈的 Push 操作中,新节点的 next 指针应指向( ),再更新栈顶指针 (B)

- A. 原栈底节点
- B. 原栈顶节点
- C. NULL
- D. 任意节点

8 栈用于括号匹配检验时,读入右括号后,若栈顶为同类左括号则( ),否则匹配失败 (C)

- A. 左括号入栈
- B. 右括号入栈
- C. 栈顶左括号出栈
- D. 清空栈

9 递归函数调用时,系统将本层参数和返回地址存入( ),递归退层时从中恢复 (B)

- A. 队列
- B. 栈
- C. 数组
- D. 链表

10 设有一个栈,元素按给定顺序依次入栈,在入栈过程中可随时执行出栈操作,其入栈序列为 a, b, c, d, e, 允许进栈退栈交替进行,但不允许连续 3 次退栈,不可能的出栈序列是 (D)

- A. b, c, d, e, a
- B. c, d, b, e, a
- C. b, c, a, e, d
- D. e, d, c, b, a

11 顺序栈 SqStack 的成员函数 Pop (ElemType &e) 的核心操作是 (B)

- A.  $e = \text{elems}[\text{count}--]$
- B.  $e = \text{elems}[--\text{count}]$
- C.  $e = \text{elems}[\text{count}++]$
- D.  $e = \text{elems}[++\text{count}]$

12 链式栈的 Clear 操作需要( ) (B)

- A. 逐个删除栈底节点
- B. 逐个删除栈顶节点
- C. 直接置 top 为 NULL
- D. 重置 count 为 0

13 入栈序列为 1, 2, ..., n, 出栈序列  $p_1 p_2 \dots p_n$  中  $p_3=4$ , 则  $p_4$  不可能的取值是 (D)

- A. 3

- B. 5
- C. 2
- D. 1

14 栈的 Traverse 操作要求从 ( ) 到 ( ) 遍历元素 (B)

- A. 栈顶; 栈底
- B. 栈底; 栈顶
- C. 任意顺序
- D. 仅遍历栈顶

15 多栈共享空间的主要目的是 (C)

- A. 提高插入速度
- B. 提高删除速度
- C. 提高空间利用率
- D. 简化操作

16 队列的核心特性是 ( ) , 允许插入的一端称为 (D)

- A. 后进先出; 队头
- B. 先进先出; 队头
- C. 后进先出; 队尾
- D. 先进先出; 队尾

17 链队列的队空条件是 ( ) , 队满条件是 ( ) (A)

- A.  $\text{front} == \text{rear}$ ; 无
- B.  $\text{front} == \text{rear}$ ; 内存不足
- C.  $\text{front} == \text{NULL}$ ;  $\text{rear} == \text{NULL}$
- D.  $\text{front} != \text{rear}$ ; 内存不足

18 顺序队列存在假溢现象, 解决该问题的最佳方案是 (C)

- A. 增大数组容量
- B. 元素前移
- C. 采用循环队列
- D. 改用链队列

19 循环队列的 maxSize 为 10,  $\text{front}=3$ ,  $\text{rear}=7$ , 队列中元素个数为 (A)

- A. 4
- B. 5
- C. 6
- D. 7

20 循环队列的队满条件是 ( ) (牺牲一个空间法) (A)

- A.  $(\text{rear} + 1) \% \text{maxSize} == \text{front}$
- B.  $\text{rear} == \text{front}$
- C.  $\text{rear} == \text{maxSize} - 1$

D.  $\text{front} == \text{maxSize} - 1$

21 循环队列的 InQueue 操作中, rear 的更新方式是 (C)

A.  $\text{rear}++$

B.  $\text{rear}--$

C.  $\text{rear} = (\text{rear} + 1) \% \text{maxSize}$

D.  $\text{rear} = (\text{rear} - 1) \% \text{maxSize}$

22 链队列删除队头元素时, 若删除后队列为空, 需将 ( ) 置为 front (B)

A. 头节点

B. 尾节点

C. 新队头节点

D. NULL

23 用两个栈 S1、S2 模拟队列, 入队操作应在 ( ) 执行, 出队操作应在 ( ) 执行 (A)

A. S1; S2 (S2 空则导入 S1 元素)

B. S2; S1 (S1 空则导入 S2 元素)

C. S1; S1

D. S2; S2

24 循环队列的 Length () 函数实现为 (B)

A.  $\text{rear} - \text{front}$

B.  $(\text{rear} - \text{front} + \text{maxSize}) \% \text{maxSize}$

C.  $\text{rear} - \text{front} + 1$

D.  $(\text{rear} - \text{front}) \% \text{maxSize}$

25 一个双端队列, 只允许从队首删除元素, 但允许从队首或队尾插入元素。元素按 a, b, c, d, e 的顺序入队完毕, 再全部出队, 下列哪个出队序列是不可能的 (C)

A. a, b, c, d, e

B. e, d, c, b, a

C. b, a, d, c, e

D. c, a, b, d, e

26 链队列的 front 指针指向 ( ), rear 指针指向 ( ) (A)

A. 头节点; 尾节点

B. 首元素节点; 尾节点

C. 头节点; 首元素节点

D. 尾节点; 头节点

27 循环队列的 OutQueue 操作中, front 的更新方式是 (C)

A.  $\text{front}++$

B.  $\text{front}--$

- C.  $\text{front} = (\text{front} + 1) \% \text{maxSize}$
- D.  $\text{front} = (\text{front} - 1) \% \text{maxSize}$

28 链队列的入队操作需创建新节点，将其链接在 ( ) 之后，再更新 rear (B)

- A. 头节点
- B. 尾节点
- C. 任意节点
- D. 队头元素节点

29 循环队列采用“标志位法”判断满空时，flag=1 表示 ( )，flag=0 表示 ( ) (A)

- A. 满；空
- B. 空；满
- C. 操作成功；操作失败
- D. 操作失败；操作成功

30 入队序列为 1, 2, 3, 4, 5，循环队列 maxSize=6，front=0，入队 5 个元素后 rear= ( )，再入队 1 个元素则 ( ) (A)

- A. 5；队满溢出
- B. 5；成功入队
- C. 4；队满溢出
- D. 4；成功入队

31 表达式求值中，操作符栈 optr 和操作数栈 opnd 的初始化状态是 (B)

- A. optr 空；opnd 空
- B. optr 压入 '='；opnd 空
- C. optr 空；opnd 压入 0
- D. optr 压入 '('；opnd 空

32 中缀表达式转换为后缀表达式时，遇到运算符  $\theta$ ，若其优先级 ( ) 栈顶运算符，则栈顶运算符弹出并输出，重复该过程后压入  $\theta$  (D)

- A. 大于
- B. 大于等于
- C. 小于
- D. 小于等于

33 后缀表达式 “3 4 + 2 \* 5 /” 的计算结果是 (A)

- A. 2.8
- B. 3
- C. 3.5
- D. 4

34 中缀表达式 “a + b \* (c - d) / e” 的后缀表达式是 (B)

- A. abc\*d-e/+

- B.  $abcd-*e/+$
- C.  $ab+cd*-e/$
- D.  $abcd-*+e/$

35 表达式求值的算符优先法中，Isp（栈内优先级）和 Icp（栈外优先级）的关系中，只有（ ）时直接压栈（B）

- A.  $Isp > Icp$
- B.  $Isp < Icp$
- C.  $Isp == Icp$
- D.  $Isp != Icp$

36 中缀表达式求值时，当 optr 栈顶为 '（' 且当前字符为 '）'，应（B）

- A. '）'入栈
- B. 弹出 '（' 并继续读取下一个字符
- C. 计算 '（' 和 '）' 之间的表达式
- D. 报错

37 后缀表达式求值时，遇到运算符则弹出（ ）个操作数进行运算，结果压栈（B）

- A. 1
- B. 2
- C. 3
- D. 4

38 中缀表达式 “ $1 + 2 * 3 - 4 / 5$ ” 转换为后缀表达式时，运算符弹出的顺序是（A）

- A.  $* / + -$
- B.  $* / - +$
- C.  $+ * - /$
- D.  $* + / -$

39 表达式转换中，栈内运算符的弹出条件是（A）

- A. 栈顶运算符优先级  $\geq$  当前运算符优先级（除 '（' 外）
- B. 栈顶运算符优先级  $>$  当前运算符优先级（除 '（' 外）
- C. 栈顶运算符优先级  $\leq$  当前运算符优先级（除 '（' 外）
- D. 栈顶运算符优先级  $<$  当前运算符优先级（除 '（' 外）

40 中缀表达式求值时，若 optr 栈顶优先级  $>$  当前运算符优先级，应（B）

- A. 当前运算符入栈
- B. 弹出栈顶运算符，从操作数栈取两个元素运算，结果压栈
- C. 直接计算当前运算符
- D. 清空操作符栈

41 链式栈的 Top 操作实现为（ ），若栈空则返回 UNDER\_FLOW（A）

- A.  $e = top \rightarrow data$

- B. `e = top->next->data`
- C. `e = elems[top]`
- D. `e = elems[count-1]`

42 循环队列的 GetHead 操作中，若队列非空，e 应赋值为 (B)

- A. `elem[rear]`
- B. `elem[front]`
- C. `elem[(front+1)%maxSize]`
- D. `elem[(rear-1)%maxSize]`

43 入栈序列为 x, y, z，出栈序列可能的个数是 (B)

- A. 4
- B. 5
- C. 6
- D. 7

44 链队列的析构函数需要 ( )，避免内存泄漏 (C)

- A. 仅释放头节点
- B. 仅释放尾节点
- C. 遍历释放所有元素节点和头节点
- D. 置 front 和 rear 为 NULL

45 循环队列 maxSize=8，front=5，rear=2，队列中元素个数为 (C)

- A. 3
- B. 4
- C. 5
- D. 6

46 中缀表达式 “ $(a + b) * c - (d - e) / f$ ” 转换为后缀表达式时，栈中运算符的最大个数是 (A)

- A. 3
- B. 4
- C. 5
- D. 6

47 栈的 Empty 操作判断条件是 ( ) (顺序栈) (A)

- A. `count == 0`
- B. `top == -1`
- C. `count == maxSize`
- D. `top == maxSize`

48 队列的 Traverse 操作应从 ( ) 到 ( ) 遍历元素 (A)

- A. 队头；队尾
- B. 队尾；队头

- C. 任意顺序
- D. 仅遍历队头

49 多栈共享数组  $S[10]$ ，栈 1 的  $Top1=3$ ，栈 2 的  $Top2=6$ ，此时栈 1 最多还能入栈（ ）个元素（A）

- A. 2
- B. 3
- C. 4
- D. 5

50 表达式求值中，当输入流结束（ $ch=''$ ）且  $optr$  栈顶为 '=' 时，运算结果存放在（B）

- A.  $optr$  栈顶
- B.  $opnd$  栈顶
- C. 输入流
- D. 临时变量

## 第 4 章

- 1 关于串的定义，下列说法错误的是 (C)
- A. 串是  $n$  ( $n \geq 0$ ) 个字符的有限连续序列， $n=0$  时为 空串
  - B. 串相等的条件是长度相等且对应位置字符完全相同
  - C. 空格串的长度为 0，与空串等价
  - D. 子串是串中任意个连续字符构成的序列，空串是所有串的子串
- 2 设串  $S$  长度为  $n$  ( $n > 0$ )，则其所有子串 (含空串和自身) 的个数为 (B)
- A.  $n(n+1)/2$
  - B.  $n(n+1)/2 + 1$
  - C.  $2^n$
  - D.  $n^2$
- 3 设串采用下标从 0 开始的顺序存储结构，子串提取操作  $\text{SubString}(S, \text{pos}, \text{len})$ ，假设允许提取空串，下列关于该操作参数合法性的说法中，正确的是 (D)
- A. 若  $\text{pos} = n$  且  $\text{len} = 0$ ，则操作非法
  - B. 若  $\text{pos} = 0$  且  $\text{len} = n + 1$ ，操作可能成功
  - C. 只要  $0 \leq \text{pos} < n$ ，无论  $\text{len}$  取何值，操作都合法
  - D. 操作合法当且仅当  $0 \leq \text{pos} \leq n$  且  $0 \leq \text{len} \leq n - \text{pos}$
- 4 关于串的“位置”定义，下列描述正确的是 (A)
- A. 子串在主串中的位置以其子串第一个字符在主串的位置表示
  - B. 字符在串中的位置从 0 开始，子串的位置从 1 开始
  - C. 同一字符在串中多次出现时，其位置必然相同
  - D. 空串在主串中的位置为 -1
- 5 设串  $A = \text{"abc"}$ ，串  $B = \text{"abc "}$  (末尾含 1 个空格)，下列判断正确的是 (D)
- A.  $A$  与  $B$  长度相等
  - B.  $A$  与  $B$  相等
  - C.  $B$  是  $A$  的子串
  - D.  $A$  是  $B$  的子串
- 6 串的  $\text{concat}$  操作中，若串  $\text{addTo}$  为空串、 $\text{addOn}$  为非空串，则操作结果是 (C)
- A. 结果为空串
  - B. 结果为  $\text{addTo}$  原串
  - C. 结果与  $\text{addOn}$  相同
  - D. 结果为  $\text{addOn} + \text{addTo}$
- 7 下列关于空串的说法，错误的是 (B)
- A. 空串长度为 0
  - B. 空串不可以作为子串
  - C. 空串无实际字符



D. 空串与任何串联接后为原串

8 设串  $S = \text{"aabbaabb"}$ ，其最长相等真前缀和真后缀的长度为 (C)

- A. 2
- B. 3
- C. 4
- D. 6

9 串操作 `Index (const String &target, const String &pattern, int pos=0)` 的参数约束中，下列必须满足的是 (A)

- A. pattern 非空且  $0 \leq \text{pos} < \text{target.Length}()$
- B.  $\text{pos} \geq 0$  且  $\text{pos} \leq \text{target.Length}()$
- C. pattern 长度  $\leq$  target 长度
- D.  $\text{pos} \geq 1$  且  $\text{pos} \leq \text{target.Length}() - \text{pattern.Length}()$

10 关于串的“真子串”（不含自身和空串），若串长为  $n$ ，则真子串个数为 (B)

- A.  $n(n+1)/2$
- B.  $n(n+1)/2 - 1$
- C.  $n(n-1)/2$
- D.  $2^n - 2$

11 C 语言中字符串的存储本质是 (B)

- A. 字符数组，以数组长度标识串长
- B. 以 `'\0'` 结束的字符数组
- C. 动态分配的字符序列，无需结束符
- D. 栈区固定长度的字符集合

12 C 语言定长顺序存储结构中，`char str[100]` 最多可存储的字符数为 (C)

- A. 100
- B. 101
- C. 99
- D. 无限制

13 关于 C 语言堆式顺序存储结构，下列说法正确的是 (A)

- A. 存储空间动态分配，适应串长变化
- B. 必须使用 `'\0'` 作为结束符
- C. 存储密度低于定长顺序存储
- D. 无需手动释放存储空间

14 串的块链存储结构中，提高存储密度的关键是 (D)

- A. 减少指针域字节数
- B. 采用单链表结构
- C. 取消尾指针
- D. 每个结点存放多个字符

15 C 语言中，字符常量 'A' 与字符串常量 "A" 的核心区别是 (C)

- A. 存储的 ASCII 码不同
- B. 字符常量可修改，字符串常量不可修改
- C. 字符串常量末尾多一个 '\0'
- D. 字符常量存储在堆区，字符串常量存储在栈区

16 下列关于 C 语言字符串存储的说法，错误的是 (C)

- A. 定长存储可通过整数记录串长，无需 '\0'
- B. 堆式存储需手动管理内存（分配 / 释放）
- C. 块链存储中每个结点仅存 1 个字符时存储密度最高
- D. 字符串常量存储在只读数据区

17 要在 C 语言字符串中包含双引号，正确的写法是 (B)

- A. "Hello"World""
- B. "Hello \"World\""
- C. 'Hello "World"'
- D. "Hello \World"

18 String 类的成员变量通常不包括 (D)

- A. char \*strVal（存储串值）
- B. int length（存储串长）
- C. 构造函数与析构函数
- D. int maxLen（预分配的最大存储容量）

19 String 类的转换构造函数不包括 (C)

- A. String (const char \*copy)
- B. String (const String &copy)
- C. String (int len, char fillChar)
- D. String (char c)

20 String 类成员函数 CStr () 的功能是 (A)

- A. 返回以 '\0' 结尾的 C 风格字符串指针，用于兼容 C 函数
- B. 将当前串转换为整数
- C. 清空串内容
- D. 返回串的长度

21 String 类的复制构造函数应实现 (B)

- A. 浅拷贝，多个对象共享同一块内存
- B. 深拷贝，为目标对象重新分配内存并复制内容
- C. 仅复制长度信息
- D. 抛出异常禁止复制

22 在实现 String 类的 Concat 操作（将当前串与另一个串拼接）时，最合理的首要步骤是 (D)

- A. 释放当前串原有的存储空间
- B. 直接将另一个串的内容追加到当前串的缓冲区末尾
- C. 调用 CStr() 获取两个串的 C 风格字符串指针
- D. 计算拼接后新串的总长度

23 String 类重载的 operator [] 的返回类型是 (C)

- A. char
- B. char \*
- C. const char &
- D. String &

24 关于 String 类的析构函数，其核心作用是 (A)

- A. 释放 strVal 指向的动态存储空间
- B. 将 length 设为 0
- C. 销毁整个 String 对象
- D. 清空 strVal 内容

25 String 类的 Empty () 函数返回 true 的条件是 (B)

- A. strVal 为 NULL
- B. length == 0
- C. 串仅含空格
- D. strVal [0] == '\0'

26 下列关于 String 类 operator == 的实现逻辑，正确的是 (D)

- A. 比较 strVal 指针地址
- B. 比较 length 是否相等
- C. 直接比较字符数组长度
- D. 基于 strcmp 函数比较串值

27 String 类构造函数 String (const char \*inString) 中，分配存储空间的大小是 (C)

- A. strlen(inString)
- B. strlen(inString)-1
- C. strlen(inString)+1
- D. 固定 256 字节

28 BF 算法的最坏时间复杂度是 (n 为目标串长，m 为模式串长) (B)

- A.  $O(n+m)$
- B.  $O(n \times m)$
- C.  $O(n)$
- D.  $O(m)$

29 KMP 算法的核心优化是 (A)

- A. 不回溯目标串指针
- B. 减少模式串比较次数
- C. 预处理目标串
- D. 降低空间复杂度

30 关于 KMP 算法的 next 数组，下列说法错误的是 (C)

- A. next 数组仅与模式串相关
- B. next[j] 表示模式串前 j 个字符的最长相等前缀后缀长度
- C. j=0 时 next[j] = 0
- D. next 数组可通过递推求得

31 设模式串 P="ababac"，其 next 数组的值为 (B)

- A. [-1, 0, 1, 2, 3, 0]
- B. [-1, 0, 0, 1, 2, 3]
- C. [0, -1, 0, 1, 2, 0]
- D. [-1, 0, 1, 2, 3, 3]

32 首尾字符串模式匹配算法的优化点是 (B)

- A. 不回溯目标串
- B. 从模式串首尾同时比较，提前发现不匹配
- C. 预处理模式串
- D. 空间复杂度为  $O(1)$

33 KMP 算法中，当目标串 S[i] ≠ 模式串 P[j] 时，下一步比较的是 (D)

- A. S[i+1] 与 P[0]
- B. S[i-j+1] 与 P[0]
- C. S[i] 与 P[j-1]
- D. S[i] 与 P[next[j]]

34 BF 算法中，当目标串 S="aaaaa"，模式串 P="aaab" 时，匹配失败的次数为 (B)

- A. 1
- B. 2
- C. 3
- D. 4

35 KMP 算法的总时间复杂度是 (A)

- A.  $O(n+m)$
- B.  $O(n \times m)$
- C.  $O(n^2)$
- D.  $O(m^2)$

36 关于首尾匹配算法的内循环逻辑，下列正确的是 (B)

- A. 仅比较模式串首部字符

- B. 同时比较首部和尾部字符，不匹配则退出
- C. 从中间向两端比较
- D. 仅比较模式串尾部字符

37 设模式串  $P = \text{"aaaaa"}$ ，其 next 数组的值为 (A)

- A.  $[-1, 0, 1, 2, 3]$
- B.  $[-1, -1, -1, -1, -1]$
- C.  $[0, 1, 2, 3, 4]$
- D.  $[-1, 0, 0, 0, 0]$

38 下列关于三种匹配算法的对比，正确的是 (C)

- A. 首尾匹配算法效率最高
- B. BF 算法空间复杂度最高
- C. KMP 算法无回溯，适用于大文件处理
- D. 首尾匹配算法预处理时间为  $O(m)$

39 KMP 算法中，模式串向右滑动的步长为 (B)

- A.  $j - \text{next}[j-1]$
- B.  $j - \text{next}[j]$
- C.  $\text{next}[j]$
- D.  $m - j$  ( $j$  为当前不匹配位置)

40 关于 KMP 算法的预处理阶段（求解 next 数组），时间复杂度是 (A)

- A.  $O(m)$
- B.  $O(n)$
- C.  $O(m^2)$
- D.  $O(n \times m)$

## 第 5 章

1 已知二维数组  $A[3 \times 4]$  (下标从 1 开始), 采用行优先顺序存储, 基地址  $\text{Loc}[1, 1]=100$ , 每个元素占 2 个存储单元, 则元素  $A[2, 3]$  的存储地址为 (C)

- A. 108
- B. 110
- C. 112
- D. 114

2 C 语言中定义二维数组  $A[b_1 \times b_2]$  (下标从 0 开始), 基地址  $\text{Loc}[0, 0]=L$ , 每个元素占  $\text{size}$  个单元, 则元素  $A[i, j]$  的地址计算公式为 (D)

- A.  $\text{Loc}[0, 0] + (i * (b_2 + 1) + j) * \text{size}$
- B.  $\text{Loc}[0, 0] + ((i - 1) * b_2 + j - 1) * \text{size}$
- C.  $\text{Loc}[0, 0] + (i * b_1 + j) * \text{size}$
- D.  $\text{Loc}[0, 0] + (i * b_2 + j) * \text{size}$

3 若下标从 1 开始计, 对于  $n$  阶对称矩阵  $A$ , 按行优先存储下三角元素到数组  $s$  中,  $s[1]$  对应  $A[1, 1]$ ,  $s[k]$  对应  $A[i, j]$ , 若  $i < j$ , 则  $k$  的计算公式为 (B)

- A.  $i * (i + 1) / 2 + j$
- B.  $j * (j - 1) / 2 + i$
- C.  $j * (j - 1) / 2 + i - 1$
- D.  $i * (i - 1) / 2 + j - 1$

4 设 4 阶下三角矩阵  $B$ , 采用压缩存储到数组  $s$  中,  $s$  的存储容量为 (D)

- A. 4
- B. 8
- C. 9
- D. 10

5 稀疏矩阵的稀疏因子  $e$  的定义为  $e = s / (m \times n)$  ( $s$  为非零元素个数), 通常认为满足以下哪个条件时称为稀疏矩阵 (A)

- A.  $e \leq 0.05$
- B.  $e \leq 0.1$
- C.  $e \leq 0.15$
- D.  $e \leq 0.2$

6 对于  $n$  阶三对角矩阵, 按行优先顺序存储非零元素, 需占用的存储单元个数为 (C)

- A.  $3n$
- B.  $3n + 1$
- C.  $3n - 2$
- D.  $2n + 1$

7 广义表  $E = (a, E)$ , 其深度为 (D)

- A. 1

- B. 2
- C. 3
- D. 无穷大

8 已知广义表  $LS=((a, b, c), (d, e, f))$ ，取出原子  $e$  的运算为 (C)

- A.  $\text{head}(\text{tail}(LS))$
- B.  $\text{tail}(\text{head}(LS))$
- C.  $\text{head}(\text{tail}(\text{head}(\text{tail}(LS))))$
- D.  $\text{head}(\text{tail}(\text{tail}(\text{head}(LS))))$

9 三元组顺序表表示的稀疏矩阵转置，简单转置算法的时间复杂度为 (B)

- A.  $O(mn)$
- B.  $O(n \times tu)$  ( $n$  为列数， $tu$  为非零元素个数)
- C.  $O(tu)$
- D.  $O(m+tu)$  ( $m$  为行数)

10 快速转置算法中，辅助数组  $\text{cpot}[\text{col}]$  的作用是 (D)

- A. 存储原矩阵每列非零元素个数
- B. 存储转置矩阵每行非零元素个数
- C. 存储原矩阵每行第一个非零元素位置
- D. 存储原矩阵每列第一个非零元素在转置矩阵中的位置

11 已知二维数组  $A[5 \times 6]$  采用列优先顺序存储(下标从 1 开始)，基地址  $\text{Loc}[1, 1]=200$ ，每个元素占 3 个单元，则  $A[4, 5]$  的地址为 (B)

- A.  $200+(4-1)*6+(5-1) \times 3$
- B.  $200+((5-1)*5+(4-1)) \times 3$
- C.  $200+((4-1)*5+(5-1)) \times 3$
- D.  $200+(5-1)*6+(4-1) \times 3$

12 对称矩阵  $A[10 \times 10]$  (下标从 1 开始) 按行优先存储下三角元素， $s[1]$  对应  $A[1, 1]$ ，则  $A[7, 3]$  对应的  $s[k]$  中  $k$  的值为 (A)

- A. 24
- B. 25
- C. 26
- D. 27

13 广义表  $L=((x, y, z), a, (u, t, w))$ ，取出原子  $t$  的运算为 (D)

- A.  $\text{head}(\text{tail}(\text{tail}(L)))$
- B.  $\text{tail}(\text{head}(\text{tail}(\text{tail}(L))))$
- C.  $\text{tail}(\text{tail}(\text{head}(\text{tail}(L))))$
- D.  $\text{head}(\text{tail}(\text{head}(\text{tail}(\text{tail}(L)))))$

14 以下关于数组存储结构的描述，错误的是 (C)

- A. 数组一般采用顺序存储，因插入删除操作极少
- B. 多维数组需通过行优先或列优先方式映射为一维存储
- C. 数组中每个元素都可以通过下标直接访问
- D. 数组的存储密度高

- C. 数组是线性表，其元素只能是简单数据类型
- D. 顺序存储的数组是随机存取结构

15 在稀疏矩阵的十字链表存储结构中，每个非零元素结点（非头结点）通常包含的链域是（C）

- A. 一个指向父结点的指针
- B. 一个指向前驱结点的指针和一个指向后继结点的指针
- C. 一个指向同行下一个非零元素的指针和一个指向同列下一个非零元素的指针
- D. 一个指向子表头结点的指针

16 已知  $n$  维数组  $A[b_1 \times b_2 \times \cdots \times b_n]$ （下标从 0 开始），基地址  $\text{Loc}[0, \cdots, 0] = L$ ，每个元素占  $\text{size}$  单元，则元素  $A[j_1, j_2, \cdots, j_n]$  的地址公式为（B）

- A.  $L + (b_2 b_3 \cdots b_n j_1 + b_3 \cdots b_n j_2 + \cdots + b_n j_{n-1} + j_n) \text{size}$
- B.  $L + (b_1 b_3 \cdots b_n j_1 + b_3 \cdots b_n j_2 + \cdots + b_n j_{n-1} + j_n) \text{size}$
- C.  $L + (b_1 b_2 \cdots b_{n-1} j_1 + b_1 \cdots b_{n-1} j_2 + \cdots + b_1 j_{n-1} + j_n) \text{size}$
- D.  $L + (j_1 j_2 \cdots j_n) \text{size}$

17 下标从 0 开始计，下三角矩阵  $A[5 \times 5]$ （上三角元素为常数  $c$ ），按行优先存储到数组  $s$  中， $s[10]$  存储的是（C）

- A.  $A[5, 5]$
- B.  $A[4, 5]$
- C.  $A[4, 0]$
- D. 常数  $c$

18 广义表  $B = (e)$ ，其表头和表尾分别为（C）

- A.  $(e)$ 、 $()$
- B.  $e$ 、 $(e)$
- C.  $e$ 、 $()$
- D.  $()$ 、 $e$

19 三元组顺序表类 `TriSparseMatrix` 的成员函数 `SetElem(int r, int c, const ElemType &v)` 的功能是（A）

- A. 设置指定位置  $(r, c)$  的元素值为  $v$
- B. 获取指定位置  $(r, c)$  的元素值存入  $v$
- C. 插入元素  $(v)$  到位置  $(r, c)$
- D. 删除位置  $(r, c)$  的元素并返回  $v$

20 快速转置算法的时间复杂度为（C）

- A.  $O(tu^2)$
- B.  $O(mn)$
- C.  $O(n+tu)$
- D.  $O(m+tu)$



21 已知广义表  $D=(A, B, C)$ ，其中  $A=()$ ， $B=(e)$ ， $C=(a, (b, c, d))$ ，则  $\text{head}(\text{tail}(D))$  为 (B)

- A.  $()$
- B.  $(e)$
- C.  $((e))$
- D.  $((a, (b, c, d)))$

22 对于 4 阶对角矩阵（仅主对角线及相邻上下对角线非零），按行优先存储非零元素， $A[3, 2]$ （下标从 0 开始）对应的存储位置  $k$  为 (D)

- A. 5
- B. 6
- C. 7
- D. 8

23 数组的基本操作不包括 (C)

- A. 存取元素
- B. 修改元素
- C. 插入元素
- D. 读取元素值

24 稀疏矩阵的三元组表中，不存储的信息是 (A)

- A. 零元素的值
- B. 非零元素的行下标
- C. 非零元素的列下标
- D. 非零元素的值

25 广义表  $C=(a, (b, c, d))$  的长度和深度分别为 (A)

- A. 2、2
- B. 2、3
- C. 4、2
- D. 4、3

26 已知二维数组  $A[4 \times 5]$ （下标从 1 开始），列优先存储，基地址  $\text{Loc}[1, 1]=100$ ，每个元素占 4 个单元，则  $A[3, 4]$  的地址为 (C)

- A.  $100 + ((4-1) * 4 + (4-1)) \times 4$
- B.  $100 + ((3-1) * 5 + (4-1)) \times 4$
- C.  $100 + ((4-1) * 4 + (3-1)) \times 4$
- D.  $100 + ((3-1) * 5 + (3-1)) \times 4$

27 下标从 0 开始计，对称矩阵  $A[6 \times 6]$  按行优先存储下三角元素， $s[k]$  对应  $A[i, j]$ ，若  $i=4, j=2$ ，则  $k =$  (B)

- A. 11
- B. 12
- C. 13
- D. 14

28 以下关于三角矩阵存储的描述，正确的是 (D)

- A. 上三角矩阵的常数元素存储在数组首位置
- B. 下三角矩阵的存储容量为  $n(n-1)/2$
- C. 三角矩阵的压缩存储无法实现随机存取
- D. 三角矩阵的非零元素存储方式与对称矩阵一致

29 广义表的深度是 (C)

- A. 表中元素的个数
- B. 表中原子元素的个数
- C. 表表达式中括号的最大嵌套层数
- D. 表中子表的个数

30 稀疏矩阵转置时，简单转置算法的缺点是 (B)

- A. 空间复杂度高
- B. 时间复杂度高 ( $O(n \times tu)$ )
- C. 无法处理非零元素过多的情况
- D. 需额外辅助数组

31 已知  $n$  维数组  $A$  的地址公式为  $LOC(j_1, j_2, \dots, j_n) = LOC(0, \dots, 0) + (b_2 b_3 \dots b_n j_1 + b_3 \dots b_n j_2 + \dots + b_n j_{n-1} + j_n) * L$ ，则该数组的存储方式为 (A)

- A. 行优先顺序
- B. 列优先顺序
- C. 链式存储
- D. 随机存储

32 十字链表中，头结点的标志域 tag 值为 (B)

- A. ATOM(1)
- B. HEAD(0)
- C. LIST(2)
- D. NULL

33 广义表  $E = (a, E)$ ，其表头的深度为 (A)

- A. 0
- B. 1
- C. 2
- D. 无穷大

34 对于 5 阶对称矩阵  $A$ ，按行优先存储下三角元素到数组  $s$  中， $s$  的长度为 (C)

- A. 10
- B. 12
- C. 15
- D. 20

35 稀疏矩阵的稀疏因子  $e=0.03$ ，说明 (A)

- A. 非零元素占总元素的 3%
- B. 零元素占总元素的 3%
- C. 非零元素个数为 3
- D. 总元素个数为 100

36 已知广义表  $LS=(a, (b, c), (d, e, f))$ ，则  $\text{tail}(\text{head}(\text{tail}(LS)))$  为 (C)

- A. (b)
- B. (b, c)
- C. (c)
- D. (d, e, f)

37 二维数组  $A[3 \times 3]$  (下标从 0 开始) 行优先存储，基地址  $LOC[0, 0]=50$ ，每个元素占 2 个单元，则  $A[2, 1]$  的地址为 (B)

- A.  $50+(1 \times 3+0) \times 2=56$
- B.  $50+(2 \times 3+1) \times 2=64$
- C.  $50+(1 \times 3+2) \times 2=60$
- D.  $50+(2 \times 2+0) \times 2=58$

38 以下关于广义表存储结构的描述，错误的是 (A)

- A. 广义表可采用顺序存储结构
- B. 引用数法广义表包含头结点、原子结点、表结点
- C. 原子结点的标志域  $\text{tag}=\text{ATOM}(1)$
- D. 表结点的指针域指向子表头结点

39 特殊矩阵的压缩存储核心目的是 (D)

- A. 提高存取速度
- B. 支持插入删除操作
- C. 简化运算逻辑
- D. 节约存储单元

40 快速转置算法中，辅助数组  $\text{num}[\text{col}]$  的功能是 (A)

- A. 统计原矩阵第  $\text{col}$  列的非零元素个数
- B. 统计转置矩阵第  $\text{col}$  行的非零元素个数
- C. 存储原矩阵第  $\text{col}$  列第一个非零元素位置
- D. 存储转置矩阵第  $\text{col}$  行第一个非零元素位置

41 广义表  $A=()$ ，其长度和深度分别为 (B)

- A. 0、0
- B. 0、1
- C. 1、0
- D. 1、1

42 若下标从 1 开始计, 对于  $n$  阶上三角矩阵, 按行优先存储到数组  $s$  中,  $s[1]$  对应  $A[1,1]$ , 则  $A[i,j]$  ( $i \leq j$ ) 对应的  $s[k]$  中  $k$  的计算公式为 (C)

- A.  $i*(2n-i+1)/2 + j - i$
- B.  $j*(j+1)/2 + i$
- C.  $(i-1)*(2n - i + 2)/2 + j - i + 1$
- D.  $j*(j-1)/2 + i$

43 稀疏矩阵的十字链表存储方式, 适用于以下哪种场景 (D)

- A. 仅需查询非零元素
- B. 非零元素个数固定
- C. 需频繁进行转置运算
- D. 需频繁进行加减运算 (非零元素位置变化大)

44 已知二维数组  $A[5 \times 4]$  (下标从 1 开始) 列优先存储, 基地址  $Loc[1,1]=200$ , 每个元素占 3 个单元, 则  $A[2,3]$  的地址为 (A)

- A.  $200 + ((3-1)*5 + (2-1)) \times 3 = 233$
- B.  $200 + ((2-1)*4 + (3-1)) \times 3 = 217$
- C.  $200 + ((3-1)*4 + (2-1)) \times 3 = 227$
- D.  $200 + ((2-1)*5 + (3-1)) \times 3 = 219$

45 广义表  $D=(A,B,C)$ ,  $A=()$ ,  $B=(e)$ ,  $C=(a,(b,c,d))$ , 则  $depth(D)$  为 (B)

- A. 2
- B. 3
- C. 4
- D. 5

46 若下标从 1 开始计, 对称矩阵  $A[7 \times 7]$  按行优先存储下三角元素,  $s[1]$  对应  $A[1,1]$ , 则  $s[20]$  对应  $A[i,j]$ ,  $i$  和  $j$  的值为 (C)

- A.  $i=6, j=4$
- B.  $i=5, j=5$
- C.  $i=6, j=5$
- D.  $i=7, j=3$

47 以下关于数组和广义表的关系, 描述正确的是 (A)

- A. 数组和广义表均可视为特殊的线性表
- B. 数组的元素只能是简单数据类型
- C. 广义表的长度一定大于深度
- D. 多维数组的维数固定, 广义表的深度固定

48 稀疏矩阵的三元组表中, 若第一个三元组为  $(4,4,6)$ , 则表示该矩阵 (B)

- A. 4 行 4 列, 6 个零元素
- B. 4 行 4 列, 6 个非零元素
- C. 6 行 6 列, 4 个非零元素
- D. 6 行 4 列, 4 个非零元素

49 广义表的引用数法存储中，头结点的 ref 域作用是 (C)

- A. 存储元素值
- B. 存储子表指针
- C. 存储访问该表的指针个数
- D. 标识节点类型

50 已知广义表  $G=((a, (b, (c, d))), (e, f), ((g), h))$ ，其深度为 (B)

- A. 3
- B. 4
- C. 2
- D. 5

## 第 6 章

1 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点、10 个度为 3 的结点、1 个度为 2 的结点、10 个度为 1 的结点，则树 T 的叶子结点个数是 (B)

- A. 41
- B. 82
- C. 113
- D. 122

2 已知一棵完全二叉树的第 6 层 (根为第 1 层) 有 8 个叶结点，则该完全二叉树的结点总数最多为 (C)

- A. 39
- B. 52
- C. 111
- D. 119

3 具有 n 个结点的完全二叉树的深度为 (D)

- A.  $\lceil \log_2(n+1) \rceil$
- B.  $\lfloor \log_2 n \rfloor$
- C.  $\log_2 n + 1$
- D.  $\lfloor \log_2 n \rfloor + 1$

4 对于任意非空二叉树，若其叶子结点数为  $n_0$ ，度为 2 的结点数为  $n_2$ ，则下列关系恒成立的是 (A)

- A.  $n_0 = n_2 + 1$
- B.  $n_0 = n_2$
- C.  $n_0 = 2n_2$
- D.  $n_0 = n_2 - 1$

5 某完全二叉树共有 256 个结点，则该树的深度为 (C)

- A. 7
- B. 8
- C. 9
- D. 10

6 若某二叉树的先序遍历序列为 ABDECFG，中序遍历序列为 DBEAFGC，则其后序遍历序列为 (A)

- A. DEBFGCA
- B. DEBFGAC
- C. DBEFACG
- D. DEBAFGC

7 下列关于满二叉树和完全二叉树的说法中，正确的是 (D)

- A. 完全二叉树一定是满二叉树
- B. 满二叉树的叶子只出现在最后两层

- C. 完全二叉树的每个非叶结点都有两个孩子
- D. 满二叉树必为完全二叉树，但反之不成立

8 用顺序存储结构按完全二叉树编号方式存储深度为  $h$  的右单支二叉树(即每个非叶结点只有右孩子)，至少需要多少存储单元？ (C)

- A.  $h$
- B.  $2h$
- C.  $2^h - 1$
- D.  $h^2$

9 在线索二叉树中， $n$  个结点的二叉链表共有多少个空指针域可用于加线索？ (D)

- A.  $n-1$
- B.  $n$
- C.  $2n$
- D.  $n+1$

10 中序线索二叉树中，某结点  $p$  无左子树且  $\text{leftTag}=1$ ，则  $p \rightarrow \text{leftChild}$  指向 (A)

- A.  $p$  的中序前驱
- B.  $p$  的中序后继
- C. NULL
- D. 根结点

11 后序线索二叉树中，若某结点是其双亲的右孩子且无右子树，则其  $\text{rightChild}$  应指向 (B)

- A. 双亲结点
- B. 后序后继 (通常是双亲)
- C. NULL
- D. 最右叶结点

12 下列哪组编码不是前缀码？ (B)

- A. (0, 10, 110, 111)
- B. (0, 1, 00, 11)
- C. (00, 01, 10, 11)
- D. (1, 01, 001, 000)

13 哈夫曼树的带权路径长度 (WPL) 等于 (C)

- A. 所有结点权值之和
- B. 所有内部结点权值之和
- C. 所有叶子结点的权值乘以其路径长度之和
- D. 树的高度乘以最大权值

14 由权值集合 {5, 7, 2, 3} 构造的哈夫曼树的 WPL 为 (C)

- A. 27
- B. 30
- C. 32

D. 33

15 有  $n$  个叶子结点的哈夫曼树，其总结点数为 (D)

A.  $2n$

B.  $2n+1$

C.  $n+1$

D.  $2n-1$

16 将森林转换为二叉树时，原森林中第二棵树的根在所得二叉树中成为第一棵树根的 (C)

A. 左孩子

B. 左子树的最右结点

C. 右孩子

D. 右子树的最左结点

17 树采用孩子兄弟表示法转换为二叉树后，原树中某结点的所有兄弟在二叉树中表现为 (B)

A. 其左子树中的结点

B. 其右子树链上的结点

C. 其祖先结点

D. 其子孙结点

18 将一棵普通树转换为二叉树，其右子树一定 (A)

A. 为空

B. 非空

C. 为完全二叉树

D. 为单支树

19 森林  $F=\{T_1, T_2, \dots, T_k\}$  对应的二叉树  $B(F)$ ，其根结点的右子树对应于 (D)

A.  $T_1$  的子树森林

B.  $T_1$  本身

C. 空

D.  $\{T_2, \dots, T_k\}$  构成的森林

20 具有  $n$  个结点的不同形态的二叉树的数目等于 (A)

A. 第  $n$  个卡特兰数

B.  $n!$

C.  $2^n$

D.  $n^n$

21 具有 4 个结点的不同二叉树的形态共有 (C)

A. 12

B. 13

C. 14

D. 15



22 具有  $n$  个结点的普通树的形态数目等于具有多少个结点的二叉树的形态数目。(B)

- A.  $n$
- B.  $n-1$
- C.  $n+1$
- D.  $2n$

23 一棵完全二叉树有 1001 个结点，其叶子结点个数为 (D)

- A. 250
- B. 500
- C. 254
- D. 501

24 在完全二叉树中，编号为  $i$  的结点 ( $i > 1$ ) 的双亲结点编号为 (A)

- A.  $\lfloor i/2 \rfloor$
- B.  $\lceil i/2 \rceil$
- C.  $i-1$
- D.  $2i$

25 完全二叉树中，若结点  $i$  有右孩子，则其右孩子编号为 (B)

- A.  $2i$
- B.  $2i+1$
- C.  $i+1$
- D.  $\lfloor i/2 \rfloor + 1$

26 先序遍历序列和后序遍历序列相同，则该二叉树满足 (A)

- A. 为空或仅含根
- B. 为满二叉树
- C. 为空或所有结点仅有左孩子或仅有右孩子 (单支)
- D. 为完全二叉树

27 若某二叉树的中序遍历结果为升序序列，则该树可能是 (D)

- A. 满二叉树
- B. 完全二叉树
- C. 哈夫曼树
- D. 二叉排序树

28 层次遍历二叉树通常借助的数据结构是 (B)

- A. 栈
- B. 队列
- C. 优先队列
- D. 双端队列

29 非递归实现中序遍历时，所需辅助空间的最大深度等于 (C)

- A. 结点总数

- B. 叶子数
- C. 树的高度
- D. 度为 2 的结点数

30 线索二叉树的主要优点是 (A)

- A. 能在  $O(1)$  时间内找到某结点在某种遍历下的前驱或后继 (若已线索化)
- B. 节省存储空间
- C. 提高插入效率
- D. 支持随机访问

31 在中序线索二叉树中, 若结点  $p$  有右子树, 则其后继是 (D)

- A.  $p$  的双亲
- B.  $p$  的右孩子
- C.  $p$  的右子树中最左下结点
- D.  $p$  的右子树中第一个被中序访问的结点 (即最左后代)

32 以下哪种遍历方式不能唯一确定一棵二叉树? (C)

- A. 先序+中序
- B. 后序+中序
- C. 先序+后序
- D. 层次+中序

33 表达式  $a+b*(c-d)-e/f$  对应的二叉树, 其后序遍历序列为 (A)

- A.  $abcd-*+ef/-$
- B.  $-+ab-cd/ef$
- C.  $a+bc-d-e/f$
- D.  $abcdef+*-/$

34 若一棵二叉树的先序序列为 ABCDEF, 中序序列为 CBAEDF, 则其高度为 (A)

- A. 3
- B. 4
- C. 5
- D. 6

35 一棵二叉树的结点按完全二叉树编号为  $1 \sim n$ , 若某结点编号为 9, 则其双亲编号为 (B)

- A. 3
- B. 4
- C. 5
- D. 6

36 在  $n$  个结点的线索二叉树中, 线索的总数为 (C)

- A.  $n-1$
- B.  $n$
- C.  $n+1$
- D.  $2n$

37 采用标准线索化规则（空指针域均存储对应线索），则先序线索二叉树中，若某结点无左子树，则其 leftChild 指向（A）

- A. 先序前驱
- B. 先序后继
- C. 中序前驱
- D. NULL

38 以下关于哈夫曼编码的说法错误的是（D）

- A. 是一种前缀编码
- B. 总编码长度最短
- C. 权值大的字符编码短
- D. 所有字符的编码长度相等

39 权值为{1,2,3,4,5}的哈夫曼树，其 WPL 为（B）

- A. 32
- B. 33
- C. 34
- D. 35

40 一棵左子树为空的二叉树进行先序线索化后，空指针的个数为（D）

- A. 不确定
- B. 0
- C. 1
- D. 2

41 树的后根遍历序列与其对应二叉树的（B）遍历序列相同。

- A. 先序
- B. 中序
- C. 后序
- D. 层次

42 森林的先序遍历序列与其对应二叉树的（A）遍历序列相同。

- A. 先序
- B. 中序
- C. 后序
- D. 层次

43 森林的中序遍历序列与其对应二叉树的（B）遍历序列相同。

- A. 先序
- B. 中序
- C. 后序
- D. 层次

44 某棵普通树（非二叉树）共有 2025 个结点，其中叶子结点 125 个。按左孩子右兄弟法将其转换为二叉树后，该二叉树中无右孩子的结点个数为（D）

- A. 125
- B. 1900
- C. 2024
- D. 1901

45 二叉树的链式存储结构中， $n$  个结点需要（C）个指针域。

- A.  $n$
- B.  $n+1$
- C.  $2n$
- D.  $n-1$

46 在二叉树的顺序存储中，编号为  $i$  的结点的左孩子编号为  $2i$  的前提是（A）

- A. 该树为完全二叉树
- B. 该树为满二叉树
- C. 该树高度  $\leq \log_2 n$
- D.  $i \leq n/2$

47 若某二叉树的叶子结点数为 50，则度为 2 的结点数为（B）

- A. 48
- B. 49
- C. 50
- D. 51

48 深度为  $k$  的满二叉树的结点总数为（D）

- A.  $2^k$
- B.  $2^{k+1}$
- C.  $2^{k-1}$
- D.  $2^k - 1$

49 以下说法正确的是（C）

- A. 任何二叉树都能唯一由其先序序列确定
- B. 线索二叉树中每个结点都有前驱和后继
- C. 哈夫曼树中不存在度为 1 的结点
- D. 完全二叉树中所有叶子都在同一层

50 设一棵非空完全二叉树  $T$  的所有叶子结点均位于同一层，且每个非叶结点都有 2 个孩子，则  $T$  是（A）

- A. 满二叉树
- B. 线索二叉树
- C. 哈夫曼树
- D. 平衡二叉树

## 第 7 章

- 1 在一个具有  $n$  个顶点的无向图中, 若其邻接矩阵为  $A$ , 则顶点  $v_i$  的度等于 (D)
  - A. 第  $i$  行中 1 的个数
  - B. 第  $i$  列中 1 的个数
  - C. 第  $i$  行与第  $i$  列中 1 的总数
  - D. 第  $i$  行 (或第  $i$  列) 中 1 的个数
- 2 若某有向图的邻接矩阵中第  $i$  行全为 0, 则说明 (B)
  - A. 顶点  $v_i$  的入度为 0
  - B. 顶点  $v_i$  的出度为 0
  - C. 图不连通
  - D.  $v_i$  是孤立点
- 3 对于一个含  $n$  个顶点和  $e$  条边的无向图, 若采用邻接表存储, 则所需存储空间为 (C)
  - A.  $O(n)$
  - B.  $O(e)$
  - C.  $O(n + e)$
  - D.  $O(ne)$
- 4 在有向图中, 若存在从任意顶点  $u$  到任意顶点  $v$  的路径, 则该图称为 (C)
  - A. 连通图
  - B. 弱连通图
  - C. 强连通图
  - D. 完全图
- 5 一个具有  $n$  个顶点的连通无向图, 其生成树必定包含 (B)
  - A.  $n$  条边
  - B.  $n - 1$  条边
  - C.  $n(n - 1)/2$  条边
  - D. 至少  $n$  条边
- 6 下列关于图的遍历的说法中, 错误的是 (C)
  - A. DFS 和 BFS 都可用于判断图是否连通
  - B. DFS 使用栈, BFS 使用队列
  - C. 非连通图只需一次 DFS 即可访问所有顶点
  - D. BFS 可用于求无权图中两点间的最短路径。
- 7 若用邻接矩阵表示图, 则深度优先搜索的时间复杂度为 (C)
  - A.  $O(n)$
  - B.  $O(n + e)$
  - C.  $O(n^2)$
  - D.  $O(e)$

8 普里姆 (Prim) 算法适用于 (B)

- A. 稀疏图
- B. 稠密图
- C. 有向图
- D. 非连通图

9 克鲁斯卡尔 (Kruskal) 算法构造最小生成树时, 为避免形成环, 通常使用 (C)

- A. 栈
- B. 队列
- C. 并查集
- D. 哈希表

10 若一个无向图有 7 个顶点, 要保证其在任何情况下都连通, 最少需要多少条边? (C)

- A. 6
- B. 15
- C. 16
- D. 21

11 在 AOE 网中, 关键路径是指 (C)

- A. 边数最多的路径
- B. 权值之和最小的路径
- C. 权值之和最大的路径
- D. 顶点数最多的路径

12 在拓扑排序中, 若最终输出的顶点数小于图中顶点总数, 则说明 (B)

- A. 图是连通的
- B. 图中存在环
- C. 图是 DAG
- D. 图是完全图

13 关键活动满足的条件是 (B)

- A. 最早开始时间大于最迟开始时间
- B. 最早开始时间等于最迟开始时间
- C. 最早发生时间小于最迟发生时间
- D. 活动持续时间为 0

14 迪杰斯特拉 (Dijkstra) 算法不能处理 (B)

- A. 带正权的有向图
- B. 带负权边的图
- C. 无向图
- D. 稠密图

15 弗洛伊德 (Floyd) 算法的时间复杂度为 (C)

- A.  $O(n)$
- B.  $O(n^2)$
- C.  $O(n^3)$
- D.  $O(n \log n)$

16 在无向完全图中，边的数量为 (C)

- A.  $n$
- B.  $n - 1$
- C.  $n(n - 1)/2$
- D.  $n(n - 1)$

17 在有向完全图中，弧的数量为 (D)

- A.  $n$
- B.  $n - 1$
- C.  $n(n - 1)/2$
- D.  $n(n - 1)$

18 若图  $G$  的邻接矩阵为对称矩阵，则  $G$  一定是 (B)

- A. 有向图
- B. 无向图
- C. 带权图
- D. 稀疏图

19 在邻接表中，求有向图某顶点的入度需 (B)

- A. 遍历其邻接链表
- B. 遍历整个邻接表
- C. 查看表头数组
- D. 无法求得

20 逆邻接表主要用于高效求解 (B)

- A. 出度
- B. 入度
- C. 度
- D. 邻接点

21 下列哪项不是图的存储结构？(D)

- A. 邻接矩阵
- B. 邻接表
- C. 十字链表
- D. 二叉链表

22 对于稀疏图，更合适的最小生成树算法是 (C)

- A. Prim (邻接矩阵)
- B. Prim (邻接表+堆)
- C. Kruskal
- D. Floyd

23 若某图的边数  $e$  远小于  $n^2$  ( $n$  为顶点数), 则该图属于 (C)

- A. 稠密图
- B. 完全图
- C. 稀疏图
- D. 连通图

24 在 Prim 算法中, 每次选择的边必须连接 (C)

- A. 两个已选顶点
- B. 两个未选顶点
- C. 一个已选与一个未选顶点
- D. 任意两个顶点

25 Kruskal 算法按什么顺序选择边? (C)

- A. 任意顺序
- B. 顶点编号顺序
- C. 权值从小到大
- D. 权值从大到小

26 一个有  $n$  个顶点的非连通无向图, 其生成森林包含的边数最多为 (B)

- A.  $n-1$
- B.  $n-k$  ( $k$  为连通分量数)
- C.  $n$
- D. 无法确定

27 在 AOV 网中, 活动之间的关系通过 (A)

- A. 边表示
- B. 顶点表示
- C. 权值表示
- D. 路径长度表示

28 拓扑排序算法中, 初始时应将哪些顶点入队? (B)

- A. 出度为 0 的顶点
- B. 入度为 0 的顶点
- C. 度为 0 的顶点
- D. 任意顶点

29 在 AOE 网中, 源点的特征是 (A)

- A. 入度为 0
- B. 出度为 0
- C. 度为 0
- D. 权值最大

30 在 AOE 网中, 汇点的特征是 (B)

- A. 入度为 0



- B. 出度为 0
- C. 度为 0
- D. 权值最小

31 事件  $v_i$  的最早发生时间  $ve(i)$  的计算依据是 (A)

- A. 最长路径
- B. 最短路径
- C. 平均路径
- D. 任意路径

32 事件  $v_i$  的最迟发生时间  $vl(i)$  的计算方向是 (B)

- A. 从源点正向
- B. 从汇点逆向
- C. 任意方向
- D. 按 DFS 顺序

33 若某活动的时间余量大于 0, 则该活动 (B)

- A. 是关键活动
- B. 不是关键活动
- C. 必须加速
- D. 无法完成

34 Dijkstra 算法中, 集合  $U$  表示 (B)

- A. 未访问顶点
- B. 已求得最短路径的终点集合
- C. 所有顶点
- D. 源点

35 Floyd 算法中,  $D(k)[i][j]$  表示 (A)

- A. 仅使用前  $k$  个顶点作为中间点的最短路径
- B. 路径长度为  $k$
- C. 从  $i$  到  $j$  经过  $k$  条边
- D. 第  $k$  次迭代的结果

36 若图  $G$  有  $n$  个顶点且边数小于  $n-1$ , 则  $G$  (B)

- A. 必连通
- B. 必不连通
- C. 可能连通
- D. 必为树

37 在无向图中, 其边数  $e$  与顶点数  $n$  的关系为 () 时, 图一定存在回路 (B)

- A.  $e \leq n-1$
- B.  $e \geq n$
- C.  $e = n$
- D.  $e = n(n-1)/2$

38 简单路径的定义是 (B)

- A. 起点与终点相同
- B. 除起点终点外无重复顶点
- C. 边数最少
- D. 权值最小

39 强连通分量是 (B)

- A. 无向图的极大连通子图
- B. 有向图的极大强连通子图
- C. 任意子图
- D. 生成树

40 以下哪种图一定没有环? (B)

- A. 完全图
- B. 树
- C. 稠密图
- D. 有向图

41 地图着色问题中, 国家之间的相邻关系最适合用哪种图表示 (B)

- A. 有向图
- B. 无向图
- C. AOV 网
- D. AOE 网

42 地图染色问题中, 若用邻接矩阵  $C[i][j] = 1$  表示国家  $i$  与  $j$  相邻, 则  $C$  必为 (A)

- A. 对称矩阵
- B. 上三角矩阵
- C. 单位矩阵
- D. 稀疏矩阵

43 在 DFS 遍历中, 若图用邻接表存储, 时间复杂度为 (C)

- A.  $O(n)$
- B.  $O(n^2)$
- C.  $O(n + e)$
- D.  $O(e)$

44 BFS 遍历可用于求解 (C)

- A. 最小生成树
- B. 关键路径
- C. 无权图最短路径
- D. 拓扑序列

45 若某无向图的邻接矩阵主对角线全为 0, 说明 (A)

- A. 无自环
- B. 无边
- C. 是完全图
- D. 是树

46 在 Kruskal 算法中，若当前边连接的两个顶点已在同一集合中，则 (B)

- A. 加入该边
- B. 跳过该边
- C. 删除该边
- D. 报错

47 Prim 算法初始时，集合 U 包含 (C)

- A. 所有顶点
- B. 空集
- C. 任选一个顶点
- D. 度最大的顶点

48 在 AOE 网中，加快非关键活动 (B)

- A. 能缩短工期
- B. 不能缩短工期
- C. 可能形成环
- D. 会破坏拓扑序

49 若图 G 是连通无向图，则其连通分量个数为 (B)

- A. 0
- B. 1
- C. n
- D. e

50 在 Dijkstra 算法中， $\text{dist}[j]$  表示 (A)

- A. 从源点到 j 的当前最短路径估计值
- B. j 的入度
- C. j 的出度
- D. j 的度

## 第 8 章

- 1 在长度为  $n$  的顺序表中进行顺序查找，若查找成功且各元素被查概率相等，则平均查找长度 ASL 为（ D ）
  - A.  $n$
  - B.  $n/2$
  - C.  $(n-1)/2$
  - D.  $(n+1)/2$
- 2 对长度为 16 的有序顺序表进行折半查找，查找一个不存在的元素，最多需比较次数为（ B ）
  - A. 4
  - B. 5
  - C. 6
  - D. 7
- 3 折半查找的判定树是一棵（ A ）
  - A. 平衡二叉树
  - B. 完全二叉树
  - C. 满二叉树
  - D. 单支树
- 4 下列哪棵二叉树**可能**是折半查找的判定树？（ A ）
  - A. 所有内部结点的左右子树高度差不超过 1，且中序遍历为  $1\sim n$
  - B. 左右子树高度差任意，但中序有序
  - C. 任意形态但关键字有序
  - D. 只要中序遍历有序即可
- 5 用折半查找法查找失败时的比较次数最多为（ C ）
  - A.  $\lfloor \log_2 n \rfloor$
  - B.  $\lceil \log_2 n \rceil$
  - C.  $\lfloor \log_2 n \rfloor + 1$
  - D.  $n$
- 6 二叉排序树中序遍历的结果是（ B ）
  - A. 逆序序列
  - B. 递增序列
  - C. 随机序列
  - D. 层次序列
- 7 由关键字序列  $\{1,2,3,4,5\}$  构造的二叉排序树，其最坏情况下的 ASL 为（ C ）
  - A. 2.2
  - B. 2.5
  - C. 3
  - D. 3.5

8 在二叉排序树中删除一个度为 2 的结点，将其用中序后继替代，之后重新插入该被删除的结点。关于所得新树与原树的关系，下列说法正确的是 (C)

- A. 新树与原树的结构一定完全相同
- B. 新树与原树的中序遍历结果不同
- C. 新树与原树的结构一定不同
- D. 无法确定两者结构关系

9 删除二叉排序树中度为 2 的结点时，通常用其 ( D ) 替代

- A. 左孩子
- B. 右孩子
- C. 父结点
- D. 中序前驱或后继

10 若在一棵 AVL 树中插入新结点导致失衡，且失衡结点 A 的左孩子 B 的右子树插入了新结点，则应进行 ( C )

- A. LL 型旋转
- B. RR 型旋转
- C. LR 型旋转
- D. RL 型旋转

11 AVL 树中每个结点的平衡因子只能取值为 ( A )

- A. -1, 0, 1
- B. 0, 1
- C. -2, -1, 0, 1, 2
- D. 任意整数

12 插入关键字序列 {5,4,2,8,6,9} 构造 AVL 树，过程中第一次发生失衡是在插入 ( B ) 之后

- A. 4
- B. 2
- C. 8
- D. 6

13 在 AVL 树中插入结点后，最多需要 ( B ) 次旋转即可恢复平衡

- A. 1
- B. 2
- C.  $\log n$
- D.  $n$

14 m 阶 B 树中，除根外的每个内部结点至少包含 ( D ) 个关键字

- A.  $\lceil m/2 \rceil$
- B.  $\lfloor m/2 \rfloor$
- C.  $\lceil m/2 \rceil + 1$
- D.  $\lceil m/2 \rceil - 1$

15 一棵高度为 3 (根为第 1 层) 的 5 阶 B 树, 最少包含的关键字个数为 ( A )

- A. 5
- B. 7
- C. 8
- D. 14

16 B 树中所有叶子结点 (失败结点) 位于 ( B )

- A. 最底层或次底层
- B. 同一层
- C. 高度相差不超过 1
- D. 任意层

17 B+ 树与 B 树的主要区别之一是 ( C )

- A. B+ 树内部结点不存储关键字
- B. B+ 树不是平衡树
- C. B+ 树的所有关键字都出现在叶子结点
- D. B+ 树不支持范围查询

18 B+ 树的叶子结点之间通常通过 ( D ) 连接以支持高效范围查询

- A. 双亲指针
- B. 哈希链
- C. 栈结构
- D. 双向链表

19 散列表的理想平均查找长度 ASL 接近 ( A )

- A. 1
- B.  $\log n$
- C.  $n$
- D. 0

20 散列函数设计的基本要求不包括 ( D )

- A. 计算简单
- B. 地址分布均匀
- C. 定义域覆盖所有关键字
- D. 必须为一一映射

21 除留余数法中, 模数  $p$  最好选择 ( B )

- A. 偶数
- B. 不大于表长的素数
- C. 表长的一半
- D. 任意合数

22 在哈希表中, 若两个关键字  $k_1 \neq k_2$ , 但  $H(k_1) = H(k_2)$ , 则称  $k_1$  和  $k_2$  为 ( )。

- A. 冲突
- B. 同义词

- C. 碰撞
- D. 哈希碰撞

23 使用链地址法处理冲突时，查找成功的平均查找长度主要取决于（ A ）

- A. 装填因子  $\alpha$
- B. 表长
- C. 关键字个数
- D. 哈希函数复杂度

24 给定关键字序列 {19,1,23,14,55,68,11,82,36}， $H(\text{key})=\text{key}\%11$ ，采用线性探测法，68 的比较次数为（ C ）

- A. 1
- B. 2
- C. 3
- D. 4

25 给定关键字序列 {19,1,23,14,55,68,11,82,36}， $H(\text{key})=\text{key}\%11$ ，采用线性探测法，构造的散列表查找成功的 ASL 约为（ B ）

- A. 2.0
- B. 2.44
- C. 2.5
- D. 3.0

26 使用链地址法，关键字 {32,40,36,53,16,46,71,27,42,24,49,64}， $H(\text{key})=\text{key}\%13$ ，查找成功的 ASL 为（ A ）

- A. 1.5
- B. 1.8
- C. 2.0
- D. 2.2

27 散列表的装填因子  $\alpha$  定义为（ D ）

- A. 表长 / 元素个数
- B. 冲突次数 / 元素个数
- C. 最大链长 / 平均链长
- D. 元素个数 / 表长

28 当  $\alpha$  接近 1 时，开放定址法的性能会（ C ）

- A. 显著提升
- B. 保持稳定
- C. 急剧下降
- D. 无法确定

29 在哈希表中查找不成功时，线性探测法的平均探测长度约为（ C ）

- A.  $1/(1-\alpha)$
- B.  $(1+1/(1-\alpha))/2$

- C.  $(1+1/(1-\alpha)^2)/2$   
D.  $\alpha/(1-\alpha)$

30 给定关键字 {7,8,11,18,9,14,30},  $H(\text{key})=(\text{key} \times 3) \bmod T$ , 要求  $\alpha=0.7$ , 则 T 应为 ( C )

- A. 7  
B. 9  
C. 10  
D. 11

31 给定关键字 {7,8,11,18,9,14,30},  $H(\text{key})=(\text{key} \times 3) \bmod T$ , 要求  $\alpha=0.7$ , 对于线性探测法, 查找成功的 ASL 为 ( D )

- A. 1  
B. 7/8  
C. 9/7  
D. 8/7

32 给定关键字 {7,8,11,18,9,14,30},  $H(\text{key})=(\text{key} \times 3) \bmod T$ , 要求  $\alpha=0.7$ , 对于线性探测法, 查找不成功的 ASL 为 ( B )

- A. 2.8  
B. 3.2  
C. 3.5  
D. 4.0

33 以下哪种方法**不适合**频繁删除的场景? ( A )

- A. 开放定址法  
B. 链地址法  
C. 公共溢出区  
D. 再哈希法

34 平方取中法适用于 ( C )

- A. 关键字分布极不均匀  
B. 关键字为字符串  
C. 关键字位数较多且分布未知  
D. 表长为素数

35 直接定址法的缺点是 ( D )

- A. 冲突率高  
B. 计算复杂  
C. 不支持删除  
D. 地址空间必须与关键字范围一致

36 动态查找表的特点是 ( B )

- A. 查找过程中不允许修改  
B. 查找过程中可插入或删除  
C. 必须使用哈希结构  
D. 只能用顺序存储



37 静态查找表适合使用 ( A )

- A. 折半查找
- B. 二叉排序树
- C. AVL 树
- D. B 树

38 判定树中, 某结点的层次等于 ( C )

- A. 该记录在表中的位置
- B. 比较次数减 1
- C. 查找该记录所需的比较次数
- D. 树的高度

39 二叉排序树的查找时间复杂度最坏为 ( D )

- A.  $O(\log n)$
- B.  $O(1)$
- C.  $O(n \log n)$
- D.  $O(n)$

40 AVL 树的高度  $h$  与结点数  $n$  的关系满足 ( A )

- A.  $h = O(\log n)$
- B.  $h = O(n)$
- C.  $h = O(\sqrt{n})$
- D.  $h = O(1)$

41 B 树主要用于 ( B )

- A. 内存高速查找
- B. 减少磁盘 I/O
- C. 实现哈希表
- D. 字符串匹配

42 在 B+ 树中, 查找操作 ( D )

- A. 可在内部结点终止
- B. 比 B 树慢
- C. 不能用于等值查询
- D. 必须到达叶子结点

43 若在 AVL 树中插入结点后, 从插入点到根路径上有多个失衡结点, 应调整 ( A )

- A. 离插入点最近的失衡结点
- B. 根结点
- C. 所有失衡结点
- D. 最深的失衡结点

44 4 阶 B 树的非根内部结点最少有 ( B ) 个关键字

- A. 0
- B. 1

- C. 2
- D. 3

45 散列函数  $H(\text{key}) = \text{key} \% 11$ ，关键字 26 和 39 的哈希地址（ B ）

- A. 分别为 3 和 5
- B. 分别为 4 和 6
- C. 分别为 5 和 7
- D. 分别为 6 和 8

46 随机探测再散列中，增量序列应（ D ）

- A. 为常数
- B. 为线性序列
- C. 为平方序列
- D. 为伪随机且覆盖全表

47 公共溢出区法的主要缺点是（ B ）

- A. 冲突处理复杂
- B. 查找效率随溢出增多而下降
- C. 不支持删除
- D. 空间浪费严重

48 在二叉排序树中插入相同关键字（假设允许），通常（ A ）

- A. 插入失败或忽略
- B. 插入到右子树
- C. 插入到左子树
- D. 替换原结点

49 折半查找要求表的存储结构为（ C ）

- A. 链表
- B. 任意顺序结构
- C. 顺序存储的有序表
- D. 索引顺序表

50 对于主关键字查找，结果（ D ）

- A. 可能为空
- B. 可能多个
- C. 取决于次关键字
- D. 唯一或不存在

## 第 9 章

- 1 在希尔排序中，若待排序序列长度为 12，采用原始增量序列，第二趟增量为 (B)  
A) 4  
B) 3  
C) 2  
D) 1
- 2 快速排序在最坏情况下的时间复杂度为  $O(n^2)$ ，该情况通常出现在 (B)  
A) 待排序序列完全随机  
B) 待排序序列已基本有序  
C) 待排序序列元素互不相同  
D) 每次划分都能将序列均分为两部分
- 3 下列排序算法中，不是原地排序的是 (C)  
A) 堆排序  
B) 快速排序  
C) 归并排序  
D) 希尔排序
- 4 对关键字序列 {48, 24, 18, 53, 16, 26, 40} 进行一趟冒泡排序后，交换次数为 (C)  
A) 3  
B) 4  
C) 5  
D) 6
- 5 堆排序中，若要实现升序排列，应构建 (B)  
A) 小顶堆  
B) 大顶堆  
C) 平衡二叉搜索树  
D) 完全二叉树即可
- 6 已知序列 {5, 8, 12, 19, 28, 20, 15, 22} 是小根堆，插入关键字 3 后，调整得到的新堆首元素为 (A)  
A) 3  
B) 5  
C) 8

D) 12

7 归并排序的稳定性来源于 (B)

- A) 使用递归实现
- B) 合并两个有序子序列时不改变相等元素的相对顺序
- C) 时间复杂度低
- D) 空间复杂度高

8 基数排序的时间复杂度为  $O(d(n + r))$ ，其中  $r$  表示 (C)

- A) 关键字位数
- B) 记录总数
- C) 基数 (如十进制中的 10)
- D) 最大关键字值

9 下列排序方法中，每一趟排序后至少有一个元素被放置到其最终位置的是 (C)

- A) 直接插入排序
- B) 希尔排序
- C) 快速排序
- D) 归并排序

10 若对  $n$  个元素进行堆排序，建堆过程的时间复杂度为 (A)

- A)  $O(n)$
- B)  $O(n \log n)$
- C)  $O(\log n)$
- D)  $O(n^2)$

11 对数组 [48, 62, 35, 77, 55, 14, 35, 98] 进行一趟快速排序 (以第一个元素 48 为枢轴)，则分区结束后，枢轴元素 48 最终所在的位置索引 (从 0 开始计) 是 (B)

- A) 2
- B) 3
- C) 4
- D) 5

12 下列排序算法中，平均性能最好的是 (C)

- A) 冒泡排序
- B) 直接插入排序
- C) 快速排序

D) 简单选择排序

13 使用挖坑法对数组 [27, 15, 18, 9, 32, 21, 10] 进行一趟快速排序（以首元素 27 为枢轴），以下哪一项是分区结束后的正确结果？（A）

- A) [10, 15, 18, 9, 21, 27, 32]
- B) [21, 15, 18, 9, 10, 27, 32]
- C) [10, 15, 18, 9, 21, 32, 27]
- D) [9, 15, 18, 10, 21, 27, 32]

14 对于外部排序，若初始归并段数为  $m$ ，采用  $k$  路归并，则归并趟数为（A）

- A)  $\lceil \log_k m \rceil$
- B)  $\lfloor \log_k m \rfloor$
- C)  $m/k$
- D)  $k \cdot m$

15 下列排序算法中，辅助空间复杂度为  $O(n)$  的是（C）

- A) 堆排序
- B) 希尔排序
- C) 归并排序
- D) 快速排序

16 假设某序列经两趟排序后变为 {11, 12, 13, 7, 8, 9, 23, 4, 5}，则该排序方法最可能是（B）

- A) 冒泡排序
- B) 插入排序
- C) 选择排序
- D) 二路归并排序

17 堆是一种（C）

- A) 二叉搜索树
- B) AVL 树
- C) 完全二叉树
- D) 线索二叉树

18 在堆排序中，输出堆顶元素后，重建堆的操作称为（B）

- A) 上滤
- B) 下滤（筛选）
- C) 旋转

D) 分裂

19 对于 TOP-K 问题（求前 K 个最大元素），最优解法是 (C)

- A) 全排序后取前 K 个
- B) 使用大顶堆
- C) 使用小顶堆维护 K 个元素
- D) 快速排序分区一次

20 LSD 基数排序的正确处理顺序是 (B)

- A) 从最高位到最低位
- B) 从最低位到最高位
- C) 按关键字大小排序
- D) 随机处理各位

21 下列排序算法中，不稳定的是 (D)

- A) 归并排序
- B) 冒泡排序
- C) 直接插入排序
- D) 希尔排序

22 快速排序的递归深度最坏情况下为 (B)

- A)  $O(\log n)$
- B)  $O(n)$
- C)  $O(n \log n)$
- D)  $O(1)$

23 直接插入排序在最好情况下的时间复杂度为 (A)

- A)  $O(n)$
- B)  $O(n \log n)$
- C)  $O(n^2)$
- D)  $O(1)$

24 希尔排序的效率主要依赖于 (B)

- A) 初始序列是否有序
- B) 增量序列的选择
- C) 关键字是否重复
- D) 是否使用递归

25 对  $n$  个元素进行简单选择排序，总共需要进行关键字比较的次数为 (A)

A)  $n(n-1)/2$

B)  $n \log n$

C)  $n$

D)  $n^2$

26 下列排序方法中，时间复杂度不受输入数据分布影响的是 (C)

A) 快速排序

B) 直接插入排序

C) 堆排序

D) 希尔排序

27 在 2 路归并排序中，第  $i$  趟归并后，每个有序子序列的长度为 (B)

A)  $i$

B)  $2^i$

C)  $2i$

D)  $n/2^i$

28 基数排序适用于 (B)

A) 浮点数排序

B) 字符串或定长整数排序

C) 任意实数排序

D) 链表结构无法使用

29 外部排序中，减少外存访问次数的关键是 (B)

A) 增大内存容量

B) 增加归并路数  $k$

C) 使用更稳定的内部排序

D) 减少初始归并段数量

30 若用堆的方法求最大 10 个元素（共 10000 个），时间复杂度约为 (D)

A)  $O(n)$

B)  $O(n \log n)$

C)  $O(k \log n)$

D)  $O(n \log k)$

31 快速排序中，若每次选取的枢轴都是最大或最小值，则递归树高度为 (B)

A)  $\log n$

B)  $n$

- C)  $n/2$
- D) 1

32 在堆的向下调整过程中，比较次数最多为 (B)

- A)  $O(1)$
- B)  $O(\log n)$
- C)  $O(n)$
- D)  $O(n \log n)$

33 对序列 {25, 13, 10, 12, 9} (大根堆) 尾部插入 18, 调整过程中元素比较次数为 (B)

- A) 1
- B) 2
- C) 3
- D) 4

34 下列排序算法中，不能保证每一趟都有元素归位的是 (D)

- A) 简单选择排序
- B) 冒泡排序
- C) 快速排序
- D) 希尔排序

35 归并排序的手动操作中，合并 [1, 3, 5] 和 [2, 4, 6] 的第一步应取 (A)

- A) 1
- B) 2
- C) 3
- D) 6

36 基数排序中，若关键字为 3 位十进制数，则需进行分配-收集的趟数为 (B)

- A) 1
- B) 3
- C) 10
- D) 1000

37 下列关于稳定性的说法正确的是 (C)

- A) 所有  $O(n \log n)$  排序都稳定
- B) 堆排序是稳定的
- C) 归并排序是稳定的



D) 快速排序是稳定的

38 在外部排序中，若内存可容纳 1000 个元素，文件含 10000 个元素，则初始归并段数为 (B)

- A) 1
- B) 10
- C) 100
- D) 1000

39 希尔排序最后一趟 ( $d=1$ ) 本质上是 (C)

- A) 冒泡排序
- B) 简单选择排序
- C) 直接插入排序
- D) 快速排序

40 快速排序的空间复杂度主要由 ( ) 决定。 (B)

- A) 数组大小
- B) 递归栈深度
- C) 关键字范围
- D) 是否原地排序

41 简单选择排序的稳定性为 (B)

- A) 稳定
- B) 不稳定
- C) 取决于实现
- D) 有时稳定

42 对  $n$  个元素进行 2 路归并排序，总比较次数上限约为 (B)

- A)  $n$
- B)  $n \log_2 n$
- C)  $n^2$
- D)  $\log_2 n$

43 LSD 基数排序中，“分配”阶段的时间复杂度为 (B)

- A)  $O(r)$
- B)  $O(n)$
- C)  $O(d)$

D)  $O(n + r)$

44 下列排序中，最适合处理“基本有序”序列的是 (C)

- A) 堆排序
- B) 快速排序
- C) 直接插入排序
- D) 基数排序

45 在堆排序中，建堆时从哪个节点开始向下调整（下标从 1 开始）？ (B)

- A) 根节点
- B) 第  $\lfloor n/2 \rfloor$  个节点
- C) 最后一个节点
- D) 第  $\lceil n/2 \rceil$  个节点

46 快速排序中，若采用随机化枢轴选择，其期望时间复杂度为 (B)

- A)  $O(n)$
- B)  $O(n \log n)$
- C)  $O(n^2)$
- D)  $O(\log n)$

47 外部排序总时间主要受 ( ) 影响。 (B)

- A) 内部排序时间
- B) 归并趟数
- C) 磁盘转速
- D) CPU 频率

48 对关键字序列 (AB, BD, ED) 进行基数排序 (LSD)，若按字母排序，基数  $r =$  (C)

- A) 2
- B) 3
- C) 26
- D) 52

49 下列排序算法中，最不适合链表结构的是 (B)

- A) 归并排序
- B) 快速排序
- C) 插入排序
- D) 冒泡排序

50 若某排序算法在任何输入下时间复杂度均为  $O(n \log n)$ ，则它不可能是 (C)

- A) 归并排序
- B) 堆排序
- C) 快速排序
- D) 以上都不是